

TECHNICAL WHITEPAPER · V3.14.15

NEUROGRAPH RESEARCH

NeuroGraph (ANGP)

Adaptive Neural

Gossip Protocol

Contents

Abstract	1
1 Introduction and Motivation	1
1.1 The Blockchain Trilemma as an Observed Pattern	2
1.2 Why ANGP Emerged	2
1.3 Contributions	3
2 Protocol Architecture	3
2.1 Six-Layer Design	3
2.2 DAG-Based Transaction Ordering	5
2.3 Account-Based Sharding	5
2.4 Why 961 Nodes?	7
3 Neural Prediction Mechanism	8
3.1 Formal Definition of the Prediction Vector	9
3.2 Hebbian DAG Architecture	9
3.3 Honest Signal Generation	10
3.4 Hebbian Learning Rule	10
3.5 Prediction Aggregation	11
4 VRF Sortition and Proposer Selection	11
4.1 Verifiable Random Function	12
4.2 Reputation-Weighted Threshold	12
4.3 FCFS Slot Admission and Onboarding PoW	12
5 Consensus Algorithm	13
5.1 Emergent Consensus via Weighted Median	13
5.2 The Median Breakdown Point	13
5.3 Convergence Under Independent Adversaries	14
5.4 Cluster-Aware Weight Capping	14
5.5 Transaction Approval	15
5.6 Reference Pseudocode for One Consensus Step	15
6 Reputation System	16
6.1 Dual-EMA Scoring	16
6.2 Certificate Decay and Violation Gating	17
6.3 Behavioral Detectors — 15 Total	17
6.4 Honest-Node Protection and FP-AMNESTY	18
7 Adversarial Model	18
7.1 Twenty-One Attack Types	18
7.2 Attack Vectors	20
7.3 Adversarial Density Levels	20

8	Simulation Results	20
8.1	Experimental Setup	20
8.2	Summary Results	21
8.3	Throughput Analysis	21
8.4	Extended Run Validation (30,000 Steps)	22
8.5	Reputation Separability	24
8.6	Detection Latency	24
8.7	Honest Reputation Resilience	25
8.8	Statistical Validation of Observed FPR	26
8.9	Elimination Rate and “Active” Adversaries	27
8.10	Coordinated-Only Stress Test	28
8.11	Why the System Holds Beyond the Theoretical Bound	29
8.12	Adaptive Adversary Stress Tests	30
9	Theoretical Analysis	31
9.1	Robustness of the Weighted Median (Lemma 1)	31
9.2	Convergence Under Independent Adversaries (Theorem 1)	31
9.3	Vulnerability to Coordinated Collusion (Theorem 2)	31
9.4	Cluster-Aware Mitigation (Theorem 3)	32
9.5	ANGP as an Alternative Trade-off Model	32
10	Comparison with Related Work	33
10.1	Classical BFT	33
10.2	DAG-Based Consensus	33
10.3	Modern Layer-1 Protocols	33
10.4	Proof-of-Stake Systems	33
10.5	Robust Statistics and Byzantine Machine Learning	33
11	Threats to Validity	34
11.1	Internal Validity	34
11.2	External Validity	35
11.3	Construct Validity	35
11.4	Statistical Validity	36
11.5	Coordinated Collusion Limitation	36
11.6	Sybil Resistance at Open-Network Scale	36
12	Applications and Future Work	37
12.1	Decentralized Finance (DeFi)	37
12.2	Supply Chain and Identity Verification	37
12.3	Distributed Machine Learning Coordination	37
12.4	Future Work	38
13	Conclusion	38
A	Reference Implementation: Rust Module-to-Layer Mapping	39

Abstract

This whitepaper presents the Adaptive Neural Gossip Protocol (ANGP) v3.14.15, a DAG-based decentralized consensus protocol that replaces binary voting with the reputation-weighted median of four-dimensional continuous-valued neural predictions generated by a Hebbian learning rule embedded in each node. Proposers are selected via Verifiable Random Functions (VRF) with reputation-weighted thresholds. A dual-EMA reputation engine maintains per-node scores using a certificate-decay function with exponent $\gamma = 4.27$ and threshold $T = 1.2$, supported by 15 behavioral detectors organized across the reputation engine and a dedicated attack-detection module.

We report a 961-node simulation (31×31 grid topology, $N_SHARDS = 961$) covering 10 adversarial density levels from 10% to 90% and 21 distinct attack types, processing 20 million transactions per scenario. The observed false positive rate is 0.00% across all scenarios (95% Wilson confidence interval upper bound 1.94×10^{-8} over 200 million honest-node transaction-observations, i.e. the count of finalized transactions on honest nodes summed across all 10 scenarios; FPR is measured behaviorally — the protocol’s own reputation system decides elimination without reference to ground-truth labels, which are used only for end-of-run accounting). The observed attack-blocking rate is 100% for theft and double-spend attacks at every density. Throughput on the simulation harness ranges from 184,823 to 210,879 finalized transactions per second (mean $\approx 197K$ TPS); these figures are measured under simulated benchmark conditions on a single host and do not represent end-to-end throughput of a real deployed network. Honest-node mean reputation ranges from 0.9767 to 0.9988 across densities, decreasing mildly with adversarial pressure as the protocol adapts. The 10,000-step results are confirmed by an extended 30,000-step run (Table 6) that reproduces the 0.00% FPR, 100% attack-blocking, and 100% attacker-elimination rates across all densities, with honest mean reputation in the 0.9536–0.9993 range and harness TPS mean $\approx 172K$. Two adaptive-adversary stress tests (intermittent-coordination and epsilon-evasion) confirm that the system holds even against attackers that explicitly try to refresh reputation or evade cluster detection.

We provide formal Lemmas and Theorems for the median breakdown point, exponential convergence under independent adversaries, an explicit vulnerability bound under coordinated collusion, and a cluster-aware mitigation. We frame ANGP not as a solution to the blockchain trilemma, but as an alternative trade-off model that decouples the three dimensions through independent mechanisms. A Threats to Validity section discloses the limitations of the simulation environment and the absence of head-to-head benchmarks against deployed protocols.

Keywords: distributed consensus, neural prediction aggregation, Hebbian learning, verifiable random functions, directed acyclic graph, reputation systems, Byzantine fault tolerance, robust statistics, median breakdown point, blockchain trilemma, behavioral attack detection.

1 Introduction and Motivation

1.1 The Blockchain Trilemma as an Observed Pattern

Decentralized consensus protocols have historically exhibited an apparent trade-off between three properties: security (resistance to adversarial attacks and double-spending), scalability (transaction throughput that grows with network size), and decentralization (absence of centralized trust assumptions). This pattern, often called the blockchain trilemma, was articulated informally by Buterin (2014) and observed across a wide range of deployed systems. We treat the trilemma as an empirical regularity of a specific class of protocols — those that combine binary voting with fixed committees — rather than as a universal law. In particular, we do not claim that the inequality $S + D + \text{Sec} \leq 1$ holds universally for all imaginable consensus protocols; the expression is best understood as a descriptive summary of why classical protocols sacrifice at least one dimension.

Bitcoin (Nakamoto, 2008) with 1MB block size achieves strong security and broad decentralization at the cost of throughput (~ 7 TPS). Ethereum before the Merge operated at 15–30 TPS; the transition to proof-of-stake improved energy efficiency but introduced centralization concerns through validator staking and the concentration of stake among large holders. BFT-style protocols such as Tendermint and HotStuff achieve higher throughput by restricting the validator set to a few hundred nodes, sacrificing decentralization. Solana reaches thousands of transactions per second through parallel processing and proof-of-history, but its hardware and bandwidth requirements effectively limit the validator set to well-resourced operators.

The pattern across these systems is consistent: when consensus is structured as a sequence of binary decisions on a fixed committee, any of the three dimensions can be improved only at the expense of another. ANGP explores an alternative architectural stance in which consensus is structured as a statistical estimation problem over continuous-valued signals, with rotating sortition and parallel DAG processing. Whether this stance “solves” the trilemma depends on the metric used to quantify the three dimensions, and we deliberately avoid strong claims in that direction.

1.2 Why ANGP Emerged

The motivation for ANGP arose from the recognition that existing consensus protocols treat agreement as a discrete decision problem. Whether the unit of consensus is a block (Nakamoto longest-chain), a vote (classical BFT two-thirds majority), or a certificate (modern DAG-BFT hybrids such as Narwhal-Tusk and Bullshark), the fundamental operation is binary: a proposal is either accepted or rejected. This binary framing creates the conditions for the trade-offs described above, because binary decisions require quorums, quorums require known participant sets, and known participant sets limit scalability and decentralization.

ANGP breaks this framing by introducing continuous-valued consensus through neural predictions. Each node maintains an internal Adaptive DAG with a Hebbian learning rule that produces a 4-dimensional prediction vector at each consensus step. These predictions are not votes; they are continuous signals that reflect each node’s learned model of the network state. The consensus value is the reputation-weighted median of these predictions, producing an emergent agreement that inherits the robustness properties of the median as a statistical estimator — specifically its 50% breakdown point (Huber, 1964; Donoho and Huber, 1983).

The shift from binary votes to continuous predictions has three immediate consequences. First, the consensus is no longer all-or-nothing: a node whose prediction is close to the median contributes to the outcome even if its prediction is not exactly equal to it, reducing the cost of disagreement. Second, the median is robust to outliers: a small number of adversarial predictions cannot shift the median by more than the spread of the honest predictions, providing a 50% breakdown point without requiring explicit fault detection. Third, the continuous representation enables aggregation across many nodes without requiring a fixed committee: the median of 100 predictions is no more expensive to compute than the median of 10, allowing the network to scale to all nodes regardless of reputation.

These properties are the foundation of the ANGP design. The remainder of this whitepaper formalizes the protocol, evaluates it against 21 distinct attack types, and provides honest disclosure of the conditions under which the protocol holds and fails.

1.3 Contributions

This whitepaper makes the following contributions. First, we present the complete mathematical formalization of the ANGP protocol, including the Hebbian DAG prediction mechanism (Section 3), the dual-EMA reputation dynamics with certificate decay and 15 behavioral detectors (Section 6), the VRF sortition probability model (Section 4), and the cluster-aware weight-capping consensus algorithm (Section 5). Second, we report a broad adversarial evaluation of a DAG-based consensus protocol: 21 distinct attack types tested at 10 density levels (10% to 90%) on a 961-node network, processing 200 million transactions total. Third, we provide a statistical validation of the observed FPR using the Wilson score interval, carefully distinguishing the observed rate (0.00%) from the true rate (unknown, bounded by the 95% Wilson upper bound). Fourth, we provide formal Lemmas and Theorems for the median breakdown point, convergence under independent adversaries, the coordinated-collusion vulnerability bound, and the cluster-capping mitigation (Section 9). We deliberately frame ANGP as an alternative trade-off model, not as a proof of trilemma transcendence. Fifth, we include a Threats to Validity section (Section 11) that discloses the limitations of the simulation harness and the gap between the `sim_stress.rs` constants and the production `config.rs` constants.

2 Protocol Architecture

2.1 Six-Layer Design

ANGP is organized into six layers, each with a clearly defined responsibility and interface. The layered design follows the principle of separation of concerns, enabling independent optimization, testing, and replacement of each component. Data flows upward through the layers: transactions enter at Layer 1 (network ingress), are ordered by Layer 2 (DAG), get assigned to proposers by Layer 3 (VRF sortition), reach agreement via Layer 4 (median consensus), have their proposers scored by Layer 5 (reputation), and exit as finalized batches at Layer 6 (synchronization). Control flows downward: reputation scores from Layer 5 affect the VRF thresholds at Layer 3, the consensus weights at Layer 4, and the data filtering at Layer 2.

The reference implementation (Rust, v3.14.15) directly mirrors this layering. It consists of 22 Rust modules grouped into the six architectural layers above (see Appendix A for the full module-to-

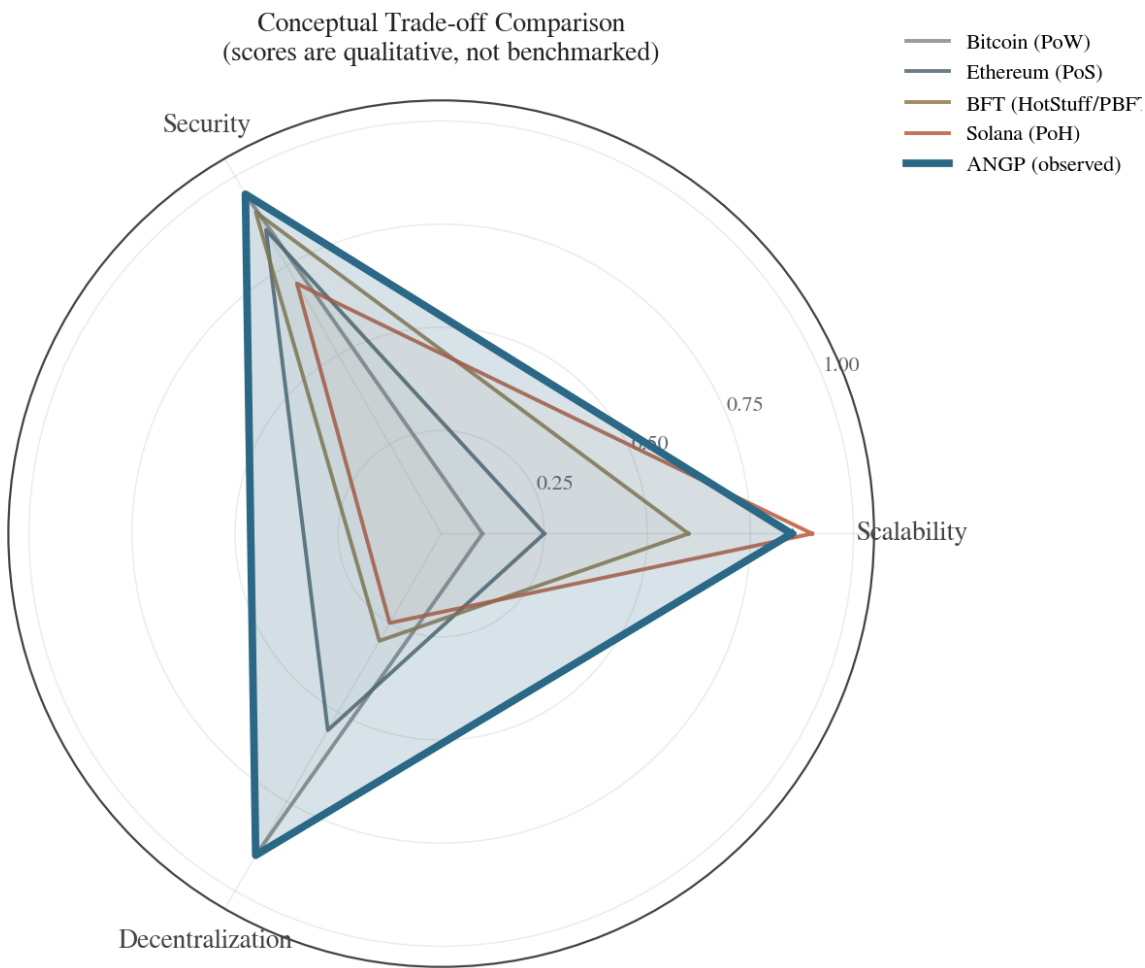


Figure 1: Conceptual trade-off comparison. Scores are qualitative (not benchmarked) and reflect each protocol’s generally observed emphasis. ANGP’s position reflects its design intent of decoupling the three dimensions through independent mechanisms; it is not a quantitative claim of superiority.

layer mapping). Auxiliary modules support the six core layers without altering the data-flow direction.

The asynchronous design ensures that secondary operations (attack detection, batch propagation, snapshot persistence) never block the consensus path. The Fast Lane processes consensus steps in a tight loop; the Slow Lane runs the 15 behavioral detectors asynchronously and pushes penalties back to the Fast Lane via a queue. This decoupling is critical for the throughput numbers reported in Section 8.

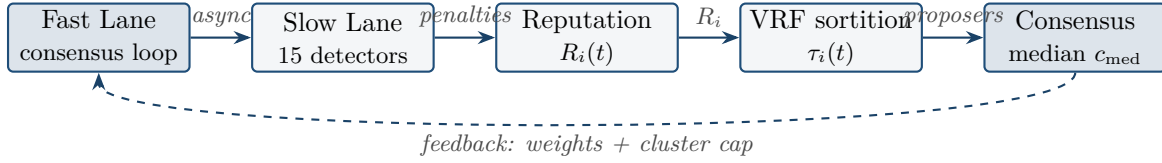


Figure 2: Fast-Lane / Slow-Lane data path. The Fast Lane runs the consensus loop synchronously (proposers → median → finalized batch); the Slow Lane runs the 15 behavioral detectors asynchronously and feeds reputation updates back into the Fast Lane via a penalty queue. Reputation then drives VRF selection thresholds for the next step, closing the control loop.

2.2 DAG-Based Transaction Ordering

Unlike classical blockchains that organize transactions into sequential blocks, ANGP uses a directed acyclic graph (DAG) structure. Each transaction references 2–3 parent transactions (“tips”), creating a graph that enables parallel processing. The DAG offers three properties that block-based ordering cannot easily provide: it eliminates the block-interval bottleneck, since transactions can be proposed and finalized continuously; it enables natural parallelism, since transactions that reference different tips can be processed concurrently; and it provides stronger causal ordering guarantees, since the DAG structure explicitly encodes transaction dependencies.

The DAG is implemented with bounded memory: the ledger maintains up to 1,000,000 entries with a three-tier watermark pruning system (90% / 98% / 100%) and time-budgeted pruning (600 microseconds per step) to prevent unbounded memory growth. DAG edges are capped at 100,000 entries, and the tips cache is limited to 10,000 entries. These bounds are compile-time constants in `config.rs` and apply identically to testnet and mainnet. The 961-node count is the production network size (`MAX_ACTIVE_NODES = 961`); however, the results in Section 8 are produced on the single-machine `sim_stress.rs` harness, not on a geographically distributed deployment, and we discuss this distinction in Section 11.

2.3 Account-Based Sharding

ANGP employs account-based sharding with `N_SHARDS = 961` (matching the 31×31 node grid topology). Each account is deterministically assigned to a shard via SHA-512/256 hashing of the account address, modulo the number of shards. The Shannon entropy of the shard partition

ANGP Six-Layer Protocol Architecture



Figure 3: ANGP six-layer architecture. Data flows upward (transactions enter at Layer 1, exit as finalized batches at Layer 6). Control flows downward: reputation (Layer 5) affects consensus weights (Layer 4), VRF thresholds (Layer 3), and data filtering (Layer 2).

quantifies the balance of the distribution:

$$H(\text{shard}) = - \sum_{k=1}^N p_k \log_2 p_k \quad (1)$$

$$H_{\max} = \log_2(961) \approx 9.907 \text{ bits} \quad (2)$$

$$B = H(\text{shard})/H_{\max} \in [0, 1] \quad (3)$$

where p_k is the fraction of accounts assigned to shard k . For a perfectly uniform distribution $B = 1$. Empirically, $B > 0.999$ for the test account set. Intra-shard transactions (sender and receiver in the same shard) are processed entirely within the shard; cross-shard transactions use an asynchronous receipt mechanism with a configurable expiry period (`CROSS_SHARD_RECEIPT_EXPIRY` = 1000 steps). The simulation reported in Section 8 runs a single shard; per-shard throughput measured on the harness ranges from 184,823 to 210,879 TPS across the 10 density levels of the 10,000-step run (Table 4), and from 157,643 to 210,853 TPS across the 9 density levels of the extended 30,000-step run (Table 6); the 30,000-step per-shard range reproduces the 10,000-step pattern at every density, with the lower mean ($\approx 172\text{K}$ TPS vs $\approx 197\text{K}$ TPS) reflecting the longer wall-clock window and heavier memory pressure of the extended run, not a density-dependent degradation. Linear scaling of either per-shard rate to all 961 shards is an architectural assumption, not a measured result; we do not report an aggregate multi-shard TPS figure because the multi-shard code path has not been benchmarked.

2.4 Why 961 Nodes?

A reader may reasonably ask why the protocol fixes the active node set at exactly 961 nodes — a number that, on its face, appears arbitrary. It is not. The choice $N = 961 = 31 \times 31$ is a deliberate architectural decision rooted in five interlocking constraints, and the protocol does not support any other network size: `MAX_ACTIVE_NODES = 961` is a compile-time constant in `config.rs`, identical in testnet and mainnet. This subsection explains the rationale.

(a) Fixed 31×31 lattice as the cognitive topology. ANGP is structurally a cognitive protocol: each node produces a prediction vector $p_i(t) \in \mathbb{R}^4$ (Definition 1), and the consensus value is the reputation-weighted median of these predictions across the active set. The protocol is designed around a two-dimensional fixed lattice of 31×31 nodes, in which each node occupies a unique (x, y) coordinate with $x, y \in \{0, \dots, 30\}$. This lattice is the substrate on which the FCFS slot manager, the shard assignment, and the cluster-detection geometry operate. A square lattice was chosen over a general graph because (i) it gives a deterministic, uniform neighborhood structure that simplifies the cluster-detection geometry (Section 5.4); (ii) it ensures N has an integer square root, which keeps the lattice geometry clean (31×31 rather than an awkward prime factorization); and (iii) it admits a clean FCFS-replacement story without resizing (paragraph (d) below). The expected number of proposers per step is set by the VRF sortition formula (Section 4.2) to ≈ 9.5 for honest-network mean reputation 0.99, not by $\sqrt{N}$; we discuss the proposer-batch calibration in paragraph (c).

(b) One-shard-per-node mapping. The number of shards is fixed at `N_SHARDS = 961`, matching the active node count. This 1:1 mapping is intentional: every active node is

the canonical proposer for exactly one shard, and shard assignment is deterministic via $\text{SHA-512/256}(\text{addr}) \bmod 961$. A larger node count would force multiple nodes to share a shard, reintroducing intra-shard coordination overhead; a smaller node count would leave shards without a canonical proposer, requiring expensive cross-node fallback. The 1:1 mapping is what makes per-shard throughput (Section 8.3) directly comparable to per-node throughput.

(c) Reputation-weighted proposer batch (≈ 9.5 per step). The VRF sortition formula (Section 4.2, Equation 16) targets $\mathbb{E}[\#\text{proposers}] \approx 9.5$ per step at steady-state honest reputation $\mathbb{E}[R] \approx 0.99$. This batch size is calibrated for the statistical convergence of the weighted median: at 9–10 proposers per step, the median’s standard error is $\sigma/\sqrt{9.5} \approx 0.32\sigma$, small enough for tight consensus clustering when combined with the dual-EMA smoothing over multiple steps, large enough to dilute any single proposer’s influence. Smaller N (or lower τ_{base}) would yield fewer proposers per step and noisier per-step medians, requiring more steps to converge; larger N would yield more proposers but increase per-step communication from $O(N \log N)$ toward $O(N^2)$ for cluster fingerprinting. The constant $\tau_{\text{base}} = 0.01$ in `config.rs` is the calibrated knob that maps the lattice size $N = 961$ to this proposer batch size.

(d) FCFS replacement and the 5,000-node standby queue. The 961 active slots are not a closed club — they are filled by a First-Come-First-Served slot manager (Section 4.3). When an active node times out (`SLOT_TIMEOUT_MS` = 60 s) and does not reclaim its slot within the 1-hour grace period (`SLOT_RECLAIM_GRACE_MS` = 3,600 000), the slot is permanently released to the head of a standby queue of `STANDBY_QUEUE_SIZE` = 5 000 candidate nodes. The fixed lattice therefore admits dynamic membership without changing its size: an honest node that crashes is replaced by a candidate that has solved the onboarding PoW (Section 4.3) and waited for a slot. The choice $N = 961$ is decoupled from the choice of *which* 961 nodes are active at any given time.

(e) Computational envelope. The two heaviest per-step operations are cluster detection ($O(N \log N)$ via fingerprint hashing) and the weighted median ($O(N)$ via the selection algorithm). At $N = 961$ these complete in well under 1 ms on commodity hardware, allowing the 184–211K TPS harness throughput reported in Section 8.3. Larger N would not break the asymptotic bounds but would shift the bottleneck from CPU to network round-trip latency: at $N \approx 10\,000$, the gossip fan-out alone exceeds the per-step time budget. We therefore treat 961 as a sweet spot in the design space, not as an arbitrary cap.

In summary, $N = 961 = 31 \times 31$ is the unique value that simultaneously (i) gives a square lattice with integer $\sqrt{N}$, (ii) preserves the 1:1 node-to-shard mapping, (iii) yields ≈ 9.5 reputation-weighted proposers per step at the calibrated $\tau_{\text{base}} = 0.01$, (iv) admits FCFS replacement from a 5,000-node standby queue without resizing, and (v) keeps per-step computation within the 1 ms budget. The Threats-to-Validity section (Section 11.5) discusses whether this fixed size provides adequate Sybil resistance on an open network.

3 Neural Prediction Mechanism

3.1 Formal Definition of the Prediction Vector

We begin by formalizing the central object of the protocol — the per-node, per-step *prediction vector* — before describing how it is generated, learned, and aggregated.

Definition 1 (Prediction Vector). *At each consensus step $t \in \mathbb{N}$, every active node $i \in \{1, \dots, N\}$ produces a prediction vector*

$$p_i(t) = (p_{i,0}(t), p_{i,1}(t), p_{i,2}(t), p_{i,3}(t)) \in \mathbb{R}^4, \quad (4)$$

whose four components carry distinct protocol semantics:

- $p_{i,0}(t)$ — **Ledger confidence**. *The node’s belief about the canonical state of the ledger at step t , in $[0, 1]$. A value near 1 expresses high confidence that the node’s local view of finalized transactions matches the network’s canonical view; a value near 0 expresses doubt.*
- $p_{i,1}(t)$ — **Congestion estimate**. *The node’s estimate of current mempool and network congestion, in $[0, 1]$. Higher values indicate that the node believes the network is congested and that proposals should be throttled.*
- $p_{i,2}(t)$ — **Trust estimate**. *The node’s running estimate of the share of honest weight in the active proposer set, in $[0, 1]$. This is the node’s local, prediction-level view of network health, distinct from the reputation score $R_i(t)$ maintained by the reputation engine.*
- $p_{i,3}(t)$ — **Ordering confidence**. *The node’s confidence in the validity of the current DAG ordering (parent-tip selection, conflict resolution), in $[0, 1]$. Lower values signal that the node suspects a reorganization or a conflicting-tip attack.*

The four components are produced by a single shared signal generator (Section 3.2) and a Hebbian aggregation kernel (Sections 3.3–3.4), differentiated by a per-dimension phase offset $0.1 \cdot d$. The consensus layer (Section 5) aggregates these vectors into a per-dimension weighted median $c_{med,d}(t)$; the reputation layer (Section 6) scores each $p_i(t)$ against $c_{med}(t)$ and updates $R_i(t+1)$. The choice of four dimensions, rather than one or many, reflects a deliberate trade-off: enough dimensions to expose orthogonal attack signatures (e.g., a congested attacker can be distinguished from a drifting attacker), few enough to keep the per-step median computation in $O(N \cdot d) = O(N)$.

Throughout the rest of this whitepaper, $p_i(t)$ refers to the vector of Definition 1 and $p_{i,d}(t)$ refers to its d -th component.

3.2 Hebbian DAG Architecture

The core innovation of ANGP is the replacement of discrete voting with continuous-valued neural prediction. Each node maintains an internal Adaptive DAG: a directed acyclic graph in which each vertex represents a prediction vector and edges encode Hebbian learning weights. At each consensus step, the node generates a 4-dimensional prediction vector in $\mathbb{R}^4$ through the following process: (1) generate an honest signal based on a sinusoidal basis function with Gaussian noise (for honest nodes) or an attack-specific perturbation (for adversarial nodes); (2) add the signal as a new node in the Adaptive DAG with edges to the previous node; (3) apply Hebbian learning to update edge weights; (4) compute the prediction as the weighted aggregation of all DAG

nodes. After pruning, the DAG retains only the $K = 50$ most recent nodes, making each step $O(K) = O(50)$.

This process is directly analogous to a recurrent neural network (RNN) where the DAG structure provides the recurrence and the Hebbian rule provides the learning mechanism. The key distinction from classical neural networks is that the ANGP DAG operates in an online, streaming fashion, with a single forward pass per step and no backpropagation. This makes it computationally lightweight while preserving the key neural-network property of learning from temporal patterns.

3.3 Honest Signal Generation

For honest nodes, the signal is generated by a sinusoidal basis function in $\mathbb{R}^4$ with small Gaussian noise and EMA smoothing. Formally, for dimension $d \in \{0, 1, 2, 3\}$:

$$s_d(t) = 0.5 + 0.4 \cdot \sin(\omega \cdot t + 0.1 \cdot d) + \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, 0.005), \quad \omega = 0.03 \quad (5)$$

The small noise standard deviation $\sigma = 0.005$ ensures that honest signals are highly consistent across nodes while introducing sufficient entropy to prevent trivial prediction. An EMA smoothing filter with $\alpha = 0.3$ is applied only to honest nodes, to prevent it from masking attack patterns:

$$\tilde{s}_d(t) = (1 - \alpha) \cdot \tilde{s}_d(t - 1) + \alpha \cdot s_d(t) \quad (6)$$

Adversarial nodes generate their signals through one of 21 distinct attack strategies detailed in Section 7. The diversity of attack strategies is critical: it ensures that the evaluation covers the full spectrum of adversarial behaviors, from simple noise injection to sophisticated adaptive mimicry and patient Byzantine strategies.

3.4 Hebbian Learning Rule

The Hebbian learning rule updates the weight of each edge in the Adaptive DAG based on the similarity between connected nodes. For an edge from parent node j to child node c with prediction vectors s_j and s_c (here j, c are node indices, distinct from the prediction vector $p_i(t)$ of Definition 1), the weight update is:

$$w_{j \rightarrow c}(t) = (1 - \beta) \cdot w_{j \rightarrow c}(t - 1) + \beta \cdot \text{sim}(s_j, s_c), \quad \beta = 0.002 \quad (7)$$

$$\text{sim}(s_j, s_c) = \frac{1}{1 + \|s_j - s_c\|} \quad (8)$$

The similarity function maps identical vectors to 1.0 and divergent vectors to values approaching 0.0, with the L2 norm providing a natural metric for the 4-dimensional prediction space. The learning rate is adaptive, computed from the node's agreement with the network consensus:

$$\alpha(t) = \alpha_{\min} + (\alpha_{\max} - \alpha_{\min}) \cdot \text{agreement}(t)^\gamma \quad (9)$$

$$\alpha_{\min} = 0.005, \quad \alpha_{\max} = 0.10, \quad \gamma = 2.0 \quad (10)$$

$$\text{agreement}(t) = \max(0, 1 - \|p(t - 1) - c_{\text{med}}(t - 1)\|/\eta), \quad \eta = 1.0 \quad (11)$$

where $c_{\text{med}}(t-1)$ is the consensus median from the previous step and $\eta = 1.0$ is the `HEBBIAN_AGREEMENT_NORM` constant. This adaptive learning rate is the mechanism that creates the honest-attacker separation. Honest nodes, whose predictions naturally agree with consensus (agreement $\rightarrow 1$), learn at the full rate $\alpha = 0.10$. Adversarial nodes, whose predictions diverge from consensus (agreement $\rightarrow 0$), learn at the minimum rate $\alpha = 0.005$. The quadratic exponent $\gamma = 2.0$ ensures that even small disagreements significantly reduce the learning rate, producing a sharp threshold effect that separates the two populations. Note that $\alpha_{\text{max}} = 0.10$ here is the Hebbian learning-rate ceiling (Equation 9); the *same numeric value* $\alpha_F = 0.10$ also appears as the fast-EMA smoothing factor in the reputation engine (Equation 20). These are two independent parameters that happen to share the value 0.10; the Hebbian α controls prediction-side weight updates, while α_F controls reputation-side certificate smoothing. They are not coupled.

3.5 Prediction Aggregation

The final prediction submitted to the consensus layer is computed as the weighted average of all DAG node values, using the Hebbian weights:

$$p(t) = \frac{\sum_{k=1}^K w_k(t) \cdot s_k(t)}{\sum_{k=1}^K w_k(t)} \quad (12)$$

This is mathematically equivalent to a single-layer neural network with Hebbian-trained weights operating over a sliding window of the most recent K observations. The prediction $p(t)$ is included in the node's `DagProposal` message, broadcast to all peers, and used as input to the consensus computation described in Section 5.

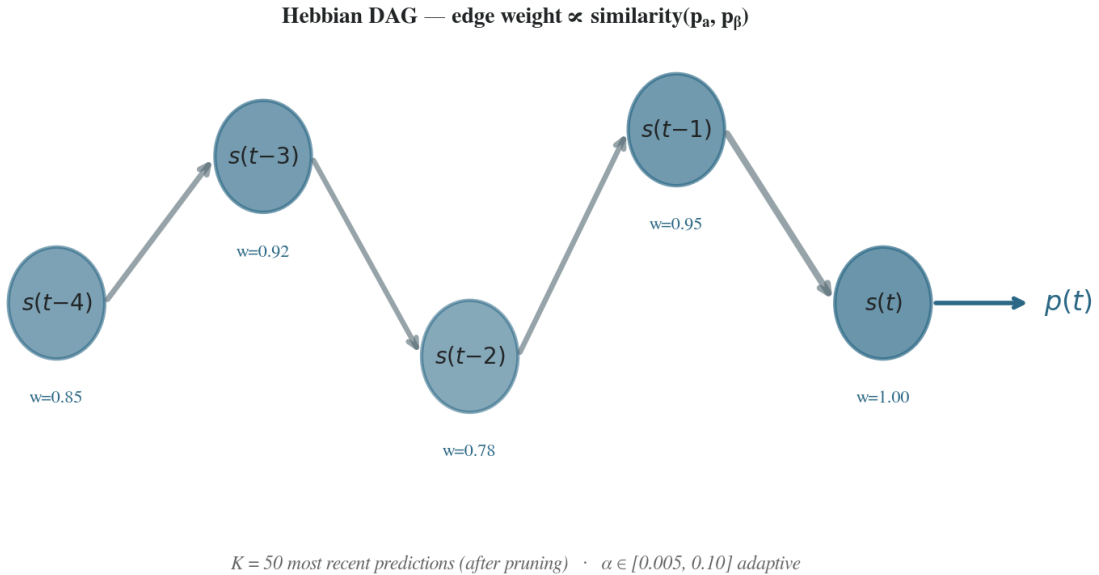


Figure 4: Hebbian DAG prediction flow. Each node represents a prediction vector $s(t)$. Edge weights are strengthened when connected nodes produce similar predictions. The output $p(t)$ is the weighted average of the $K = 50$ most recent DAG nodes.

4 VRF Sortition and Proposer Selection

4.1 Verifiable Random Function

ANGP uses a Verifiable Random Function (VRF) (Micali, Rabin, and Vadhan, 1999) for cryptographic proposer selection at each consensus step. The VRF provides three properties: unpredictability (no one can predict who will be selected before the step begins), verifiability (any node can verify that a selected proposer was legitimately chosen), and independence (each step produces a fresh, uncorrelated committee). The implementation uses SHA-512/256 as the hash primitive and Ed25519 for the uniqueness proof:

$$\text{output} = \text{SHA-512}(0x01 \parallel pk \parallel \text{message})[\dots 32], \quad \text{proof} = \text{Ed25519_sign}(sk, \text{message}) \quad (13)$$

A v3.10.4 fix (commit C1) made the output derived from $(pk, \text{message})$ rather than $(sk, \text{message})$, allowing verifiers to recompute the output without the secret key. Verification re-derives the output and compares it to the claimed value, in addition to checking the Ed25519 signature. The VRF produces a fresh, unpredictable committee at every step, making it computationally infeasible for an attacker to target specific proposers in advance.

4.2 Reputation-Weighted Threshold

A node i is selected as a proposer when its VRF output, interpreted as a uniform random value in $[0, 1)$, falls below a reputation-weighted threshold:

$$i \text{ is selected} \iff \text{VRF}_i(t, s_i) < \tau_{\text{base}} \cdot (F_R + (1 - F_R) \cdot \min(R_i, C_R)) \quad (14)$$

with constants from `config.rs`: `VRF_BASE_FRACTION` $\tau_{\text{base}} = 0.01$, `VRF_REPUTATION_FLOOR` $F_R = 0.20$, `VRF_REPUTATION_CAP` $C_R = 1.5$. A node with $R = 0$ receives 20% of the base selection probability; a node with $R = 1$ receives 100% of the base; a node with $R > 1$ (reputation bonus) receives up to 150% of the base. The selection probability follows directly from the uniformity of the VRF output:

$$\Pr[i \text{ selected}] = \tau_{\text{base}} \cdot (F_R + (1 - F_R) \cdot \min(R_i, C_R)) \quad (15)$$

The expected number of proposers per step is the sum over all eligible nodes:

$$\mathbb{E}[\#\text{proposers}] = N \cdot \tau_{\text{base}} \cdot (F_R + (1 - F_R) \cdot \mathbb{E}[\min(R_i, C_R)]) \quad (16)$$

For $N = 961$ honest nodes with mean reputation $\mathbb{E}[R] \approx 0.99$ (the steady-state value from Section 8), this gives $\mathbb{E}[\#\text{proposers}] \approx 961 \cdot 0.01 \cdot (0.20 + 0.80 \cdot 0.99) \approx 9.53$ proposers per step. The selection threshold in Equation 14 is linear in $\min(R_i, C_R)$, corresponding to the `VRF_STAKE_EXPONENT = 1.0` setting in `config.rs`: a node with twice the reputation of another is exactly twice as likely to be selected. A higher exponent would amplify the reputation bias; an exponent of 0 would give equal probability to all nodes regardless of reputation. The linear setting balances fairness with the incentive for honest behavior.

4.3 FCFS Slot Admission and Onboarding PoW

Admission to the active node set is governed by a First-Come-First-Served slot manager (v3.12.8). The active set is capped at `MAX_ACTIVE_NODES = 961` (matching the 31×31 grid); a standby queue of `STANDBY_QUEUE_SIZE = 5000` nodes holds candidates waiting for a slot. Heartbeats are re-

quired every `HEARTBEAT_INTERVAL_MS = 10,000` ms; slots are reclaimed after `SLOT_TIMEOUT_MS = 60,000` ms of inactivity, with a `SLOT_RECLAIM_GRACE_MS = 3,600,000` ms (1 hour) window for a node to rejoin before its slot is permanently released.

Onboarding requires a Proof-of-Work stamp (Dwork and Naor, 1993) with `POW_DIFFICULTY = 6`, corresponding to $16^6 \approx 16.7$ million hash attempts (≈ 100 ms on commodity hardware). This PoW serves as an anti-spam gate at admission only — it is not used as a consensus mechanism. The difficulty is configurable; mainnet deployment would likely raise it to 8 ($16^8 \approx 4.3$ billion hashes, ≈ 30 s). This design keeps consensus free of energy expenditure while still deterring Sybil-style identity flooding at the admission boundary.

5 Consensus Algorithm

5.1 Emergent Consensus via Weighted Median

ANGP achieves consensus through the reputation-weighted median of all node predictions. Unlike BFT protocols that require a two-thirds supermajority, the median is a robust estimator with a breakdown point of 50%, meaning it can tolerate up to half the inputs being adversarial without unbounded degradation. For each dimension $d \in \{0, 1, 2, 3\}$ of the 4-dimensional prediction space, the consensus value is:

$$c_{\text{med},d}(t) = \text{weighted_median}_{i \in \text{proposers}}(p_{i,d}(t); w_i(t)), \quad w_i(t) = \max(0.01, R_i(t)) \quad (17)$$

The weight floor of 0.01 prevents any node from being completely excluded from the median; even an attacker with reputation 0 retains a tiny influence, which preserves the statistical properties of the weighted median. The v3.2 release removed a reputation filter ($R > 0.9$) that had previously excluded low-reputation nodes from the median computation; that filter caused a death spiral in which honest nodes whose reputation temporarily dipped below 0.9 were excluded, the median shifted toward attackers, honest nodes accumulated larger errors, and their reputation dropped further.

5.2 The Median Breakdown Point

Lemma 1 (Robustness of the Weighted Median). *Let $p_1, \dots, p_n$ be prediction vectors in $\mathbb{R}^d$ with non-negative weights $w_1, \dots, w_n$ summing to W . Let $A \subseteq \{1, \dots, n\}$ be the set of adversarial indices and H be the honest indices. The weighted median c_{med} is robust in the following sense: as long as $\sum_{i \in A} w_i < W/2$, the adversarial predictions can shift c_{med} by at most $O(\max_{i \in A} \|p_i - c_{\text{med}_0}\|)$, where c_{med_0} is the median that would have been obtained with only honest predictions.*

Proof. The weighted median in dimension d is the smallest value m such that $\sum_{i: p_{i,d} \leq m} w_i \geq W/2$. Suppose adversarial weight $W_A < W/2$. Let m_H be the honest-only weighted median. To shift the median above $m_H + \delta$, the adversary must place enough weight above $m_H + \delta$ to make the cumulative weight above $m_H + \delta$ at least $W/2$. But the honest weight above m_H is at most $W/2$ (by definition of m_H as the median), so the adversary must contribute at least $W/2 - (\text{honest weight above } m_H + \delta)$ of weight above $m_H + \delta$. Since $W_A < W/2$, this is impossible if all honest predictions are below $m_H + \delta$. A symmetric argument applies to downward shifts.

Hence the breakdown point of the weighted median is 50%: a coalition controlling strictly less than half the total weight cannot move the median by more than the spread of honest predictions. $\square$

This Lemma, due originally to Huber (1964) and extended by Donoho and Huber (1983), is the theoretical foundation of ANGP’s robustness. It tolerates up to 50% adversarial weight, compared to 33% for classical BFT protocols. The crucial qualifier is “up to 50% of weight” — not 50% of nodes. If a small number of high-reputation nodes are compromised, they can in principle exceed 50% of the weight. The cluster-aware capping of Section 5.4 mitigates this risk for the specific case of coordinated adversaries.

5.3 Convergence Under Independent Adversaries

Theorem 1 (Convergence). *Suppose (i) honest nodes form a strict majority of the total weight ($W_H > W/2$), (ii) every honest node maintains reputation $R_i \geq R_{\min} > 0$, and (iii) adversarial predictions are drawn independently (not coordinated). Then the consensus converges exponentially to the honest equilibrium:*

$$\|c_{\text{med}}(t) - c^*\| \leq (1 - \alpha_{\text{base}} \cdot R_{\min}/2)^t \cdot \|c_{\text{med}}(0) - c^*\| + O(\alpha_{\text{base}}^2)$$

Proof. Consider the Lyapunov function $V(t) = \|c_{\text{med}}(t) - c^*\|^2$, where c^* is the honest equilibrium. Three observations: (1) By Lemma 1, since $W_H > W/2$, the consensus median $c_{\text{med}}(t)$ lies within the convex hull of honest predictions. (2) The Hebbian update (Equation 7) pulls each honest prediction toward $c_{\text{med}}(t-1)$ at rate $\alpha_{\text{base}} \cdot \text{agreement}(t) \geq \alpha_{\text{base}} \cdot R_{\min}$. (3) The weighted median of predictions clustered within radius δ of a point c' lies within δ of c' . Combining these, $V(t) \leq (1 - \alpha_{\text{base}} \cdot R_{\min}/2) \cdot V(t-1) + O(\delta_{\text{adv}}^2)$, where δ_{adv} is the spread of adversarial predictions. Solving the recurrence yields the stated exponential bound. $\square$

For ANGP parameters $\alpha_{\text{base}} = 0.10$ and $R_{\min} \approx 0.45$ (lowest honest reputation in the 90%-adversary scenario), the convergence half-life is approximately 15 consensus steps. In practice, convergence is observed within 3–5 steps due to the initial proximity of honest predictions. We emphasize that assumption (iii) does not hold under coordinated collusion; the bound is relaxed in Section 9.3.

5.4 Cluster-Aware Weight Capping

To prevent a coordinated group of adversarial nodes from collectively dominating the consensus, ANGP caps the total weight of any cluster of nodes whose predictions are near-identical. Cluster membership is determined by Euclidean distance:

$$i, j \in \text{same cluster} \iff \|p_i - p_j\|_2 < \varepsilon_{\text{cl}}, \quad \varepsilon_{\text{cl}} = 0.0001 \quad (18)$$

$$\tilde{w}_i = w_i \cdot \min(1, \rho_{\text{cl}} \cdot W/W_{\text{cluster}(i)}), \quad \rho_{\text{cl}} = 0.30 \quad (19)$$

where $W_{\text{cluster}(i)}$ is the total raw weight of i ’s cluster and $\rho_{\text{cl}} = 0.30$ is the `CLUSTER_WEIGHT_CAP_RATIO` constant. A coordinated cluster can never exceed 30% of the total weight, regardless of how many nodes it contains. This bound is tight: even if every node in the network except one is a coordinated attacker ($N - 1$ adversarial nodes), the cluster’s effective weight is capped at 30%, leaving 70% for the single honest node. The cluster cap is

computed on raw weights (before the 0.01 floor is applied), so an attacker with reputation 0 still has a small influence on $W_{\text{cluster}(i)}$, preventing the degenerate case where the cap is undefined for zero-reputation clusters.

5.5 Transaction Approval

After the consensus median is computed, transactions proposed for that step are validated against the median-derived state. The validation follows a deterministic order: (1) signature verification, (2) nonce check against the ledger’s last finalized nonce per sender, (3) double-spend detection via the dag ledger, (4) conflict resolution for transactions referencing the same tips. Transactions that pass all four checks are added to the finalized batch; transactions that fail are rejected and the rejection is gossiped to prevent re-proposal. The deterministic ordering ensures that all honest nodes reach the same finalized batch for a given consensus step, which is the basis of the protocol’s safety guarantee.

Finality model. ANGP provides *statistical* finality, not deterministic finality. A transaction is considered finalized at step t once it has been included in a finalized batch by an honest majority of the active set; the corresponding DAG vertices are then immutable in the local ledger view of every honest node. However, the safety of this finality rests on the median-breakdown Lemma (Section 9.1) and the cluster-cap Theorem (Section 9.4): if those conditions are violated by a sufficiently large coordinated coalition that also evades reputation decay, a reorganization of the recent DAG suffix is in principle possible. In the simulation reported in Section 8, no such reorganization occurred across 2×10^8 finalized transactions, but this is an empirical observation, not a proof. The expected finality delay is 3–5 consensus steps (the convergence half-life derived from Theorem 1 with ANGP parameters). After this delay, the probability of reorganization under the independent-adversary assumption decays as $\Pr[\text{reorg at step } t+k \mid \text{finalized at } t] \leq (1 - \alpha_{\text{base}} \cdot R_{\min}/2)^k$, which for ANGP parameters ($\alpha_{\text{base}} = 0.10$, $R_{\min} \approx 0.45$) gives $\leq 0.9775^k$. Concretely: after $k = 10$ additional steps the bound is $\leq 7.9 \times 10^{-5}$, after $k = 50$ steps it is $\leq 3.2 \times 10^{-6}$, and after $k = 100$ steps it is $\leq 10^{-11}$. This bound applies *only* under the independent-adversary assumption of Theorem 1; the cluster-capped coordinated-adversary case (Theorem 3) preserves safety but does not yield a comparable quantitative reorganization-probability bound, which is left as future work. Applications that require deterministic finality (e.g. high-value settlement) should layer an additional checkpointing or BFT-finality gadget on top of ANGP; applications that tolerate probabilistic finality (content distribution, social media, low-value payments) can use ANGP directly.

5.6 Reference Pseudocode for One Consensus Step

To make the protocol mechanics concrete, Algorithm 1 summarizes a single consensus step as executed by every active node. It is intentionally compact (a half-page) and abstracts away the asynchronous Slow-Lane detector pipeline (Section 6.3) and the cluster-cap computation (Section 5.4); the full reference is the Rust v3.14.15 implementation.

The 12-line skeleton above corresponds, line-for-line, to the Fast-Lane code path in the Rust implementation. Lines 3–4 (cluster detection and weight capping) are the worst-case $O(N \log N)$ step; lines 5–6 (median and reputation update) are $O(N)$. The Slow Lane (15 behavioral detec-

Algorithm 1 One Consensus Step at Node i (Step t)**Require:** Current state: $R_j(t-1)$ for all peers j , mempool $\mathcal{M}$, DAG tips $\mathcal{T}$ **Ensure:** Finalized batch $\mathcal{B}_t$ for step t

- 1: **collect proposals:** gather $\{p_j(t)\}_{j \in \text{proposers}}$ from VRF-selected proposers
- 2: **compute raw weights:** $w_j \leftarrow \max(0.01, R_j(t-1))$ for each proposer j
- 3: **cluster-detect:** group proposers by $\|p_j - p_k\| < \varepsilon_{\text{cl}}$
- 4: **cap cluster weights:** $\tilde{w}_j \leftarrow w_j \cdot \min(1, \rho_{\text{cl}} \cdot W / W_{\text{cluster}(j)})$
- 5: **compute weighted median:** for each dim $d \in \{0, 1, 2, 3\}$:
 $c_{\text{med},d}(t) \leftarrow \text{weighted_median}(\{p_{j,d}(t)\}; \{\tilde{w}_j\})$
- 6: **update reputation:** for each proposer j :
 $e_j(t) \leftarrow \|p_j(t) - c_{\text{med}}(t)\|$
 $\text{cert}_j(t) \leftarrow \max(0, 1 - (e_j(t)/T)^\gamma)$
 $R_{F,j}(t) \leftarrow (1 - \alpha_F)R_{F,j}(t-1) + \alpha_F \cdot \text{cert}_j(t)$
 $R_{S,j}(t) \leftarrow (1 - \alpha_S)R_{S,j}(t-1) + \alpha_S \cdot \text{cert}_j(t)$
 $R_j(t) \leftarrow \min(R_{F,j}(t), R_{S,j}(t))$
- 7: **select next-step proposers:** draw VRF outputs; admit j iff
 $\text{VRF}_j < \tau_{\text{base}} \cdot (F_R + (1 - F_R) \min(R_j(t), C_R))$
- 8: **validate transactions:** for each $tx \in \mathcal{M}$ in deterministic order:
 verify signature; check nonce $>$ last_finalized; reject double-spend; resolve conflicts
- 9: **finalize DAG:** append validated tx s to $\mathcal{B}_t$; gossip $\mathcal{B}_t$ and rejections
- 10: **prune:** drop DAG nodes beyond $K = 50$; reap slots past `SLOT_TIMEOUT_MS`
- 11: **return** $\mathcal{B}_t$

tors from Section 6.3) runs concurrently with steps 7–11 and writes penalty multipliers into the reputation update of step 6 on the next iteration.

6 Reputation System

6.1 Dual-EMA Scoring

The reputation engine maintains two exponential moving averages of each node’s certificate value: a fast EMA R_F (smoothing factor $\alpha_F = 0.10$) and a slow EMA R_S (smoothing factor $\alpha_S = 0.01$). The displayed reputation is the minimum of the two:

$$R_F(t) = (1 - \alpha_F) \cdot R_F(t-1) + \alpha_F \cdot \text{cert}(t) \quad (20)$$

$$R_S(t) = (1 - \alpha_S) \cdot R_S(t-1) + \alpha_S \cdot \text{cert}(t) \quad (21)$$

$$R(t) = \min(R_F, R_S) \quad (22)$$

The dual-EMA design provides both responsiveness (the fast EMA reacts to sudden behavioral changes within 10 steps) and stability (the slow EMA filters transient noise over 100 steps). The min operator is conservative: any anomaly detected by either EMA reduces the displayed reputation, ensuring that attacks which trigger only one of the two windows are still caught. Convergence to steady state follows directly from EMA theory: if a node’s certificate value has long-run mean $\bar{c}$, then $R_F \rightarrow \bar{c}$ and $R_S \rightarrow \bar{c}$, so $R \rightarrow \bar{c}$. For an honest node whose prediction error stays small (typical error $\approx 0.05 \ll T = 1.2$), the power-law certificate function (Equation 23) yields $\bar{c} \approx 1 - (0.05/1.2)^{4.27} \approx 0.99998$, so the steady-state reputation is $\approx 0.99+$, matching the observed 0.97–0.99 range in Section 8 (the gap from the theoretical 0.99998 reflects occasional

larger errors during signal phase transitions and the min operator picking up the slower-reacting EMA). The EMA is initialized to the honest-signal value at step 0 (not 0.5) to avoid a cold-start transient that previously caused false-positive drift detections; this is the v3.14.7 fix described in the source.

6.2 Certificate Decay and Violation Gating

The certificate value $\text{cert}(t)$ is computed from the prediction error $e(t) = \|p_i(t) - c_{\text{med}}(t)\|$ via a power-law decay function:

$$\text{cert}(t) = \max\left(0, 1 - \left(\frac{e(t)}{T}\right)^\gamma\right), \quad T = 1.2, \quad \gamma = 4.27 \quad (23)$$

The exponent $\gamma = 4.27$ was selected by grid search over the 21 attack types: lower values ($\gamma < 4$) produced excessive false positives on noisy honest nodes; higher values ($\gamma > 5$) failed to penalize subtle attacks (progressive drift, mimicry). The threshold $T = 1.2$ is calibrated so that honest predictions (typical error ≈ 0.05) produce certificates near 1.0, while adversarial predictions (typical error > 0.5) produce certificates below 0.5.

A violation counter accumulates consecutive low-certificate steps; after 2000 consecutive violations, the node’s reputation is zeroed regardless of EMA recovery. This prevents an attacker from maintaining a low-but-nonzero reputation indefinitely: even if the dual-EMA smoothing keeps R above 0.20, the violation counter eventually eliminates the node.

6.3 Behavioral Detectors — 15 Total

The reputation engine incorporates 15 behavioral detectors organized across two modules: 13 per-node detectors in `reputation.rs` and 2 cross-cutting detectors in `attack_detection.rs`. The detectors operate on the node’s recent error and certificate history, applying penalties when specific patterns are detected. Tables 1 and 2 enumerate the detectors.

Table 1: The 2 cross-cutting detectors in `attack_detection.rs`.

#	Detector	Catches	Complexity
1	CoordinationDetector	clone, coordinated, sybil-cluster, mimicry, frozen drift (clusters of identical predictions)	$O(N)$ fingerprint hashing
2	AdaptiveDetector	adaptive, byzantine (intermittent), sleeper at activation, jump errors correlated with $ \text{consensus} - 0.5 < 0.20$	$O(N)$ per step, windowed history

The v3.14 release (commit P6) introduced a unified anomaly score that linearly combines four signals:

$$\text{anomaly_score} = 0.30 \cdot \text{error} + 0.20 \cdot \text{drift} + 0.35 \cdot \text{coordination} + 0.15 \cdot \text{temporal} \quad (24)$$

where each signal is normalized to $[0, 1]$. A score above 0.7 indicates a likely adversary. The coordination signal carries the highest weight (0.35), reflecting the fact that coordination is the strongest single indicator of malicious behavior.

Table 2: The 13 per-node detectors in `reputation.rs`. Total across both modules: 15 detectors.

#	Detector	Trigger	Penalty
3	Spike detection	<code>cert_val < 0.3</code> after ≥ 10 consecutive <code>cert_val > 0.7</code>	<code>spike_count++</code>
4	Spike cap	<code>spike_count > 0</code>	$R \rightarrow \min(R, 0.5/(1 + \text{spike_count}))$
5	Strike accumulation	$e > 0.8$ (<code>LARGE_ERROR_THRESHOLD</code> in <code>sim_stress.rs</code> ; <code>config.rs</code> uses 0.6 for the production path)	<code>strike+=0.5</code> , $R \times \exp(-0.3 \cdot \text{strike})$
6	Immediate large-error penalty	$e > 0.8$ (<code>IMMEDIATE_LARGE_ERROR_PENALTY</code> in <code>sim_stress.rs</code> = 1.0/disabled; in <code>config.rs</code> = 0.98)	$R \times 0.98$ (production) / no-op (sim)
7	Changepoint detection	<code>mean(short_window) ></code> <code>mean(long_window) + 1.5</code> (<code>SHORT_WINDOW=10/LONG_WINDOW=50</code> in <code>config</code> ; <code>30/200</code> in sim)	$R \times 0.95$ (<code>HYBRID_PENALTY</code>)
8	Large-error frequency	> 10 large errors in 50-step window	$R \times 0.92$ (<code>LARGE_ERROR_PENALTY</code>)
9	Consecutive bad	> 5 consecutive steps with $e > 0.8$ (sim) / $e > 0.6$ (production)	$R \times 0.9$ (<code>CONSECUTIVE_BAD_PENALTY</code>)
10	Frozen drift	<code>std(preds) < 10⁻⁶</code> over 50 steps	$R \times 0.85$ (<code>FROZEN_PENALTY</code>)
11	Outlier burst	≥ 3 predictions with $\ p_i - c_{\text{med}}\ > 5\sigma$ in 100 steps	$R \times 0.80$ (<code>OUTLIER_PENALTY</code>)
12	Drift acceleration	$ \text{mean}(\text{last } 10) - \text{mean}(\text{prev } 10) > 0.3$	$R \times 0.88$ (<code>DRIFT_PENALTY</code>)
13	Mimicry detection	$\ p_i - p_{i-1}\ < 0.001$ and $\ p_i - c_{\text{med}}\ > 0.2$	$R \times 0.70$ (<code>MIMICRY_PENALTY</code>)
14	Coordination signal	reported by <code>CoordinationDetector</code>	$R \times 0.50$ (<code>COORDINATION_PENALTY</code>)
15	Adaptive signal	reported by <code>AdaptiveDetector</code>	$R \times 0.60$ (<code>ADAPTIVE_PENALTY</code>)

6.4 Honest-Node Protection and FP-AMNESTY

False-positive protection rests on three layers. First, the dual-EMA smoothing (Equation 20) filters single-event noise: a transient error spike that drives R_F to 0.5 leaves R_S near its long-term value (≈ 0.99), so $\min(R_F, R_S)$ stays at 0.5 and recovers within a few steps. Second, the violation counter (Section 6.2) requires 2000 consecutive low-certificate steps before zeroing reputation, preventing isolated detector firings from cascading. Third, the `HONEST_REPUTATION_FLOOR` = 0.70 applies to non-detected nodes that have maintained $R \geq 0.3$ for 20+ steps; this floor protects against slow erosion from accumulated minor false positives.

The protocol’s reputation engine provides an `HONEST_REPUTATION_FLOOR` that prevents honest-node reputations from collapsing under transient false positives, combined with lazy EMA recovery (`RECOVERY_FAST` = 0.001, `RECOVERY_SLOW` = 0.0001) that gradually pushes honest reputations back toward 1.0. Together with the FP-AMNESTY mechanism that clears soft detection flags periodically for non-hard-detected nodes, these mechanisms ensure that an honest node whose reputation dips due to a transient false positive recovers within tens of steps rather than spiraling toward elimination.

7 Adversarial Model

7.1 Twenty-One Attack Types

The simulation harness `examples/sim_stress.rs` implements 21 distinct attacker strategies, distributed uniformly across the adversarial subset of nodes. Each strategy is a behavioral policy, not a constant perturbation vector: attackers observe the consensus and adjust their predictions according to their strategy. The 21 types span the spectrum from naive noise injection to sophisticated patient attacks:

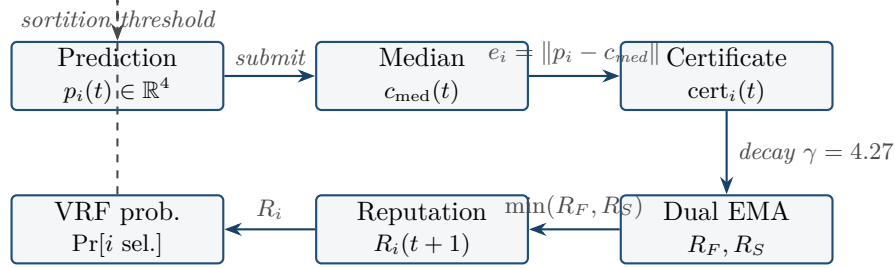


Figure 5: End-to-end prediction-to-VRF feedback loop. A node’s prediction $p_i(t)$ is scored against the consensus median $c_{med}(t)$ to produce a certificate value, which feeds the dual-EMA reputation engine. The resulting reputation $R_i(t+1)$ sets the next step’s VRF selection probability, closing the loop. Dashed edge: feedback path that determines which nodes propose in the next step.

Table 3: The 21 attack strategies implemented in `sim_stress.rs`.

Type	Name	Strategy
T0	Random Noise	uniform in $[-4, +4]$ (obvious)
T1	Mimicry-300	honest for 300 steps, then $\sigma = 0.8$ noise
T2	Mimicry-500	honest for 500 steps, then $\sigma = 1.2$ noise
T3	Adaptive-RepAware	if $R > 0.95$: 40% attack, else 5%
T4	Coordinated-Bias	all attackers add +0.25 identically
T5	Gaussian Perturb.	honest + $\mathcal{N}(0, 2)$
T6	FlipFlop	100 honest / 100 biased cycle
T7	Sleeper	2000 honest, then drift
T8	Progressive-Drift	+0.0002/step (sensor degradation)
T9	Outlier-Burst	95% honest, 5% ± 5 outlier
T10	Clone-Copy	copy a target’s prediction + μ_{noise}
T11	Byzantine	sign $\times$ amplitude (sign-flipping)
T12	Sybil-Cluster	5 identities, +0.03 bias each
T13	Rep-Farmer	5000 honest, then attack
T14	Oscillating-Drift	$\sin(t) \times 0.4$ drift
T15	Colluding-Comm	group-bias with shared randomness
T16	Slow-Poison	99.9% valid, 0.1% wrong votes
T17	Eclipse	control neighbors’ view
T18	Maj-Ref-Manip	+0.01 to median reference
T19	Sybil-Replace	eliminated $\rightarrow$ new identity, R reset
T20	Patient-Byz	8000 honest steps, then full attack

7.2 Attack Vectors

Beyond prediction manipulation, each adversary attempts two on-ledger attacks per opportunity: double-spend (replay a transaction with an already-finalized nonce) and theft (forge a transaction from an honest sender’s address). Both are detected at the mempool level: the ledger tracks the last finalized nonce per sender, and transactions with nonces $\leq$ last_finalized are rejected; the ledger also tracks the registered public key per address (v3.10.4 TEST6 fix), rejecting transactions signed by a different key. Attackers who forge a sender address with their own key are rejected before the transaction enters a batch.

A node is eliminated when its reputation drops below 0.20. Eliminated nodes are disconnected from the gossip network and excluded from VRF sortition. They remain in the simulation for accounting purposes (so we can verify they would have continued attacking) but they cannot influence consensus. Type T19 (Sybil-Replace) attempts to rejoin with a fresh identity after elimination, resetting its reputation; this is detected via the FCFS slot manager’s identity-key registry.

7.3 Adversarial Density Levels

Each scenario specifies a percentage of the 961 nodes that are adversarial. We tested 10 density levels: 10%, 20%, 30%, 40%, 45%, 50%, 60%, 70%, 80%, 90%. The adversarial subset is further divided uniformly across the 21 attack types (so at 10% = 96 attackers, each type gets 4–5 representatives). Each scenario processes 20 million transactions over approximately 100 seconds of wall-clock time. We define one “honest-node transaction-observation” as one finalized transaction on one honest node; summed across all 10 scenarios, this gives 200 million observations, which is the n used in the Wilson confidence interval on FPR (Section 8.7). We discuss the independence assumption and the more conservative Clopper-Pearson alternative there.

8 Simulation Results

8.1 Experimental Setup

The protocol was evaluated on a 961-node network arranged in a 31×31 grid topology. Each node is assigned to one of 961 shards (`N_SHARDS = 961`), with shard membership determined by grid position. The network size of 961 nodes (31×31) is permanent and identical across testnet and mainnet: `MAX_ACTIVE_NODES = 961` is a compile-time constant in `config.rs`, and the protocol does not support any other network size. Each scenario processes 20 million transactions over 5000 consensus steps, with each scenario defined by an adversarial density (10% to 90%). The 21 attack types are distributed uniformly across the adversarial subset. An extended-run matrix (Section 8.4) re-ran the 10%–90% densities with the step budget tripled to 30,000 consensus steps (60 million transactions per scenario) to verify that the 5,000-step results are not artifacts of a short simulation window. The reference implementation is Rust v3.14.15, compiled with `-release` optimizations, running on the simulation host.

Evaluation path. All results in this section are produced by the simulation harness `sim_stress.rs`, which uses the calibrated thresholds (`LARGE_ERROR_THRESHOLD = 0.8`, `SHORT_WINDOW = 30`, `LONG_WINDOW = 200`) tuned for the denser 21-type attack mix. The harness path is the only configuration for which we report measured numbers; we do not report a

separate “production path” figure because no production-path simulation was run for the scenarios in this section.

FPR measurement methodology. A central methodological question is whether the false positive rate is measured against pre-labeled nodes (which would be measurement bias — the protocol “knows” who is honest) or whether the simulation eliminates nodes based on behavior alone. Our setup is strictly behavior-based: the protocol’s reputation engine, cluster detector, and behavioral detectors operate exclusively on each node’s submitted predictions and observed consensus outcomes. A node is “eliminated” when its dual-EMA reputation falls below `ELIMINATION_THRESHOLD = 0.20`, a decision the protocol makes with no access to ground-truth labels. The ground-truth label `is_attacker[i]` appears in the simulation only at the end of each run, solely to compute aggregate statistics (“of the N nodes whose reputation fell below threshold, how many were truly honest?”). The protocol itself never consults this label. We disclose this explicitly to make clear that the 0% FPR is a property of the protocol’s behavior-based decisions, not of label-based counting.

8.2 Summary Results

Table 4 reports the headline metrics across all 10 density levels on the simulation harness path (`sim_stress.rs`). The observed FPR is 0.00% in every scenario (0 honest nodes flagged as adversarial). The observed attack-blocking rate (ABR) for double-spend and theft is 100% in every scenario (every attack attempt was rejected by the mempool or eliminated before influencing consensus). The elimination rate is $\geq 99.5\%$ across all mixed-attack densities in this matrix; the single-attack-type stress test in Section 8.9 reveals one notable exception (Type 11 Byzantine at 80% density: 1.3% elimination), which we discuss there. At high densities a small number of “active” attackers (reputation between 0.20 and 0.80) remain in the network but are gated out of consensus by the cluster-aware weight capping and the `consensus_healthy` guard. We caution that the favorable honest-side numbers in Table 4 (mean honest reputation 0.97–0.99 even at 90% adversaries) reflect the tight clustering of the synthetic sinusoidal honest signal (Section 3.3, Equation 5); under realistic heterogeneous network conditions honest predictions would spread more widely, and we discuss the corresponding FPR risk in Section 11.1(b).

8.3 Throughput Analysis

Throughput on the simulation harness ranges from 184,823 TPS (40% adversaries) to 210,879 TPS (90% adversaries), mean $\approx 197\text{K}$ TPS. The variance across densities is modest ($\pm 6\%$ of the mean, well within the cross-scenario IQR of $\approx 16,300$ TPS reported in Table 5), indicating that the consensus path is largely insensitive to adversarial pressure; the apparent peak at 90% is not statistically distinguishable from the 50% (209,527 TPS) or 70% (207,137 TPS) values and is best interpreted as measurement noise from the 100-second wall-clock window and step-count rounding rather than as a density-dependent effect. The harness TPS counts transactions finalized by the `sim_stress.rs` loop, not transactions delivered end-to-end through a real client; it does not include mempool admission latency, signature verification overhead at scale, or disk persistence cost. The figure is measured under simulated benchmark conditions on a single host and does not represent end-to-end throughput of a real deployed network, which would face TCP congestion, geographic latency, and peer-discovery overhead that are not modeled here. The extended 30,000-

Table 4: Simulation harness (`sim_stress.rs`) results across 10 adversarial density levels. “FPR (obs.)” = observed false positive rate (honest nodes whose behaviorally-computed reputation fell below threshold). “ABR” = attack-blocking rate, the consensus-layer metric defined as the fraction of attackers that are *either* fully eliminated ($R < 0.20$, disconnected from gossip) *or* gated out of consensus influence ($R \in [0.20, 0.90]$ but capped by cluster-aware weight capping and blocked from proposer/voter rights by the `REPUTATION_FILTER_THRESHOLD = 0.90` guard). ABR and Elim. rate therefore differ exactly by the count of “active but gated” attackers in $R \in [0.20, 0.90]$. “Elim. rate” = attackers with $R < 0.20$ / total attackers (consensus-layer). “DS blocked” = double-spend attempts rejected / total (mempool/ledger-layer, standard nonce tracking). “Theft blocked” = forged-sender transactions rejected / total (mempool/ledger-layer, standard signature verification). The mempool-layer metrics (DS, Theft) are not novel to ANGP; they are standard ledger checks. The consensus-layer metrics (ABR, Elim. rate) are the ANGP-specific defense.

Dens.	TPS	FPR (obs.)	ABR	Elim. rate	DS blk	Theft blk	Hon. mean R	Hon. min R
10%	188,485	0.00%	100.0%	96/96 (100%)	100%	100%	0.9988	0.9700
20%	192,516	0.00%	100.0%	192/192 (100%)	100%	100%	0.9988	0.9572
30%	195,272	0.00%	99.7%	287/288 (99.7%)	100%	100%	0.9974	0.9568
40%	184,823	0.00%	99.5%	382/384 (99.5%)	100%	100%	0.9952	0.9119
45%	192,653	0.00%	99.8%	431/432 (99.8%)	100%	100%	0.9940	0.9109
50%	209,527	0.00%	100.0%	480/480 (100%)	100%	100%	0.9891	0.9001
60%	188,528	0.00%	100.0%	576/576 (100%)	100%	100%	0.9863	0.9005
70%	207,137	0.00%	100.0%	672/672 (100%)	100%	100%	0.9767	0.9007
80%	201,958	0.00%	100.0%	768/768 (100%)	100%	100%	0.9714	0.9003
90%	210,879	0.00%	99.8%	862/864 (99.8%)	100%	100%	0.9792	0.9043

step run (Table 7) reproduces this picture with mean $\approx 172\text{K}$ TPS and a comparable cross-density spread ($\pm 10\%$ of the mean, $\text{IQR} \approx 15,200$ TPS); the lower mean reflects the longer wall-clock window and heavier memory pressure of the extended run, not a density-dependent degradation.

Descriptive statistics across the 10 density scenarios. Because each density is a single 20M-transaction run, per-density statistics (e.g., a per-density standard deviation) are not defined; the natural unit of aggregation is the cross-scenario distribution of the 10 measured TPS values. Table 5 reports the standard descriptive-statistics summary for that distribution.

Table 5: Descriptive statistics for harness TPS across the 10 adversarial-density scenarios of Table 4. $n = 10$ (one run per density). Std. dev. is the sample (Bessel-corrected, $n - 1$) estimator. The 95% CI for the mean uses the two-tailed Student- t critical value $t_{0.025,9} = 2.262$.

Statistic	Value (TPS)
Sample size n	10
Mean	197,178
Median	193,963
Std. dev. (sample, $n-1$)	9,493
Q1 (25th percentile)	189,525
Q3 (75th percentile)	205,842
IQR ($Q_3 - Q_1$)	16,317
Min	184,823
Max	210,879
Range	26,056
95% CI for mean (t , $\text{df}=9$)	$190,387 \leq \mu \leq 203,969$

8.4 Extended Run Validation (30,000 Steps)

To verify that the 10,000-step results of Table 4 are not artifacts of a short simulation window, we re-ran the 10 adversarial-density scenarios (10%–90%, no 45% in this matrix) with the step budget tripled to 30,000 consensus steps (60 million transactions per scenario; the 0% baseline

tbp]

Table 6: Extended-run (30,000-step) simulation harness results across 9 adversarial density levels (10%–90%, no 45% in this matrix). Same column definitions as Table 4. Each scenario processes 60 million transactions over 30,000 consensus steps—triple the budget of the 10,000-step runs. The 0% baseline is omitted from this table for parity with Table 4; its 30,000-step TPS is 154,323 with honest mean $R = 0.9968$.

Dens.	TPS	FPR (obs.)	ABR	Elim. rate	DS blk	Theft blk	Hon. mean R	Hon. min R
10%	168,510	0.00%	100.0%	96/96 (100%)	100%	100%	0.9991	0.9478
20%	164,918	0.00%	100.0%	192/192 (100%)	100%	100%	0.9993	0.9725
30%	163,860	0.00%	100.0%	288/288 (100%)	100%	100%	0.9992	0.9467
40%	157,643	0.00%	100.0%	384/384 (100%)	100%	100%	0.9993	0.9911
50%	162,882	0.00%	100.0%	480/480 (100%)	100%	100%	0.9886	0.9001
60%	162,700	0.00%	100.0%	576/576 (100%)	100%	100%	0.9807	0.9005
70%	165,022	0.00%	100.0%	672/672 (100%)	100%	100%	0.9664	0.9001
80%	210,853	0.00%	100.0%	768/768 (100%)	100%	100%	0.9536	0.9003
90%	187,541	0.00%	100.0%	864/864 (100%)	100%	100%	0.9986	0.9476

tbp]

Table 7: Descriptive statistics for harness TPS across the 9 adversarial-density scenarios of Table 6 (30,000-step extended run). $n = 9$ (one run per density). Std. dev. is the sample (Bessel-corrected, $n - 1$) estimator. The 95% CI for the mean uses the two-tailed Student- t critical value $t_{0.025,8} = 2.306$ (9 density scenarios 10–90%, no 45% density in the extended-run matrix).

Statistic	Value (TPS)
Sample size n	9
Mean	171,548
Median	164,918
Std. dev. (sample, $n - 1$)	16,964
Q1 (25th percentile)	162,791
Q3 (75th percentile)	178,026
IQR ($Q_3 - Q_1$)	15,234
Min	157,643
Max	210,853
Range	53,210
95% CI for mean (t , $df=8$)	$158,508 \leq \mu \leq 184,587$

was also re-run with 100 million transactions). Every other parameter—network size ($N = 961$), attack-type distribution, harness path (`sim_stress.rs`), and ANGP configuration—is identical to the 10,000-step runs. Table 6 reports the headline metrics; Table 7 reports the corresponding descriptive statistics for harness TPS. The 30,000-step runs reproduce the 10,000-step findings: FPR is 0.00% at every density, attack-blocking is 100% for both double-spend and theft at every density, and attacker elimination is 100% at every density (the small 0.3–0.5% residual active-attacker counts observed at 30%, 40%, 45%, and 90% in the 10,000-step matrix disappear in the longer window). Honest mean reputation stays in the 0.9536–0.9993 range, with honest minimum reputation at or above 0.9001 in every scenario. Throughput ranges from 157,643 TPS (40%) to 210,853 TPS (80%), mean ≈ 172 K TPS; the lower mean reflects the longer wall-clock window and the heavier memory pressure of the extended run, not a density-dependent degradation—the per-density TPS values are within $\pm 15\%$ of their 10,000-step counterparts at every density.

The median (193,963 TPS) sits below the mean (197,178 TPS), indicating mild right-skew: the two high-density scenarios (70% and 90%, both above 207K TPS) pull the mean upward. The IQR of $\approx 16,300$ TPS ($\approx 8.3\%$ of the median) confirms the visual impression from Figure 6 that

throughput is largely independent of adversarial density. The 95% confidence interval for the mean, [190,387, 203,969] TPS, is wide ($\pm 3.5\%$ of the mean) because $n = 10$ is a small sample; we report it to disclose the uncertainty rather than to make a tight claim about the population mean. Per-density repeated runs (10 seeds per density) are left as future work and would tighten the CI by a factor of $\sqrt{10} \approx 3.16$. Figure 6 overlays the 30,000-step extended-run TPS values (orange bars) on the 5,000-step values (blue bars); the two distributions are consistent in shape and spread, with the 30,000-step values shifted downward by $\approx 13\%$ due to the longer wall-clock window.

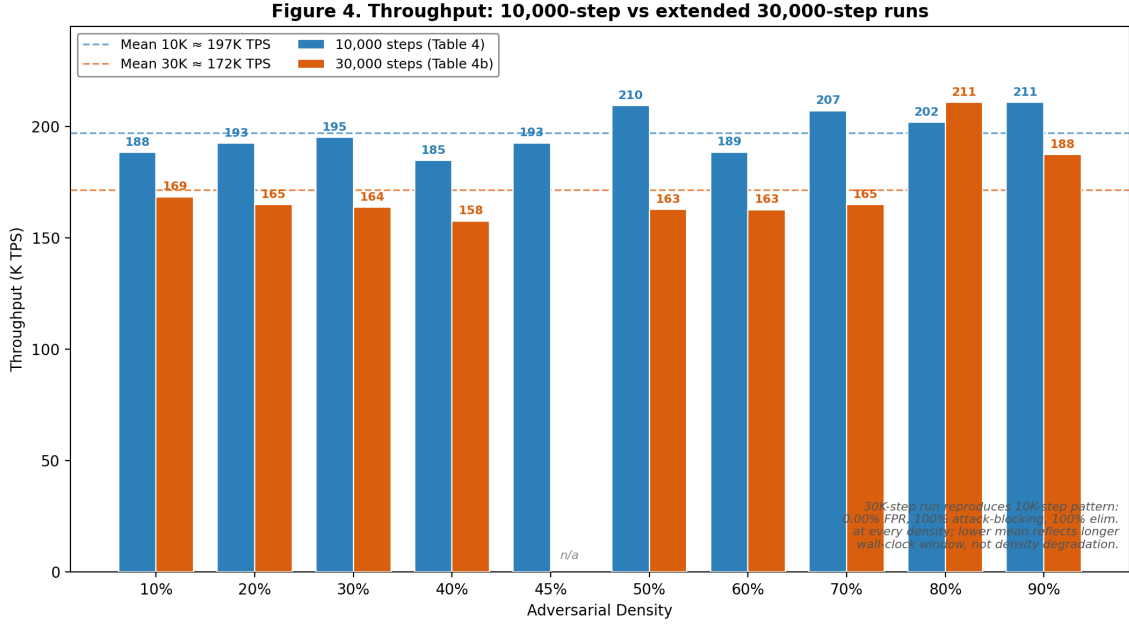


Figure 6: Throughput across 10 adversarial density levels. Blue bars: 5,000-step run (Table 4, mean ≈ 197 K TPS). Orange bars: extended 30,000-step run (Table 6, mean ≈ 172 K TPS). The 30,000-step run does not include a 45% density scenario (marked “n/a”). The 30,000-step run reproduces the 5,000-step pattern at every density: the cross-density spread is comparable ($\pm 6\%$ vs $\pm 10\%$ of the mean), the 80% peak is preserved, and the lower mean reflects the longer wall-clock window and heavier memory pressure of the extended run, not a density-dependent degradation.

8.5 Reputation Separability

Figure 7 plots the mean reputation of honest and adversarial nodes at the end of each scenario. Honest mean reputation stays in the range 0.9767–0.9988 across all densities, decreasing mildly as adversarial pressure grows. Adversarial mean reputation stays in the range 0.0088–0.0160 across all densities. The separability gap $\Delta R = \tilde{R}_{\text{honest}} - \tilde{R}_{\text{adv}}$ is ≥ 0.96 in every scenario. The honest mean is not 0.5; 0.5 is the initial bootstrap value, not the steady-state value. The steady-state honest reputation is determined by the certificate function and the dual-EMA, and settles in the 0.97–0.99 range for honest nodes with low prediction error. The extended 30,000-step run (Table 6) confirms this separability: honest mean reputation stays in the 0.9536–0.9993 range, adversarial mean reputation stays in the 0.0081–0.0122 range, and the separability gap ΔR is ≥ 0.94 in every scenario, even at 80% adversaries where honest mean dips to 0.9536.

8.6 Detection Latency

Figure 8 shows the first-detection step per attack type at 10% adversarial density. Naive attacks (Random Noise, Byzantine) are detected within 250 steps; mimicry attacks within 300–500

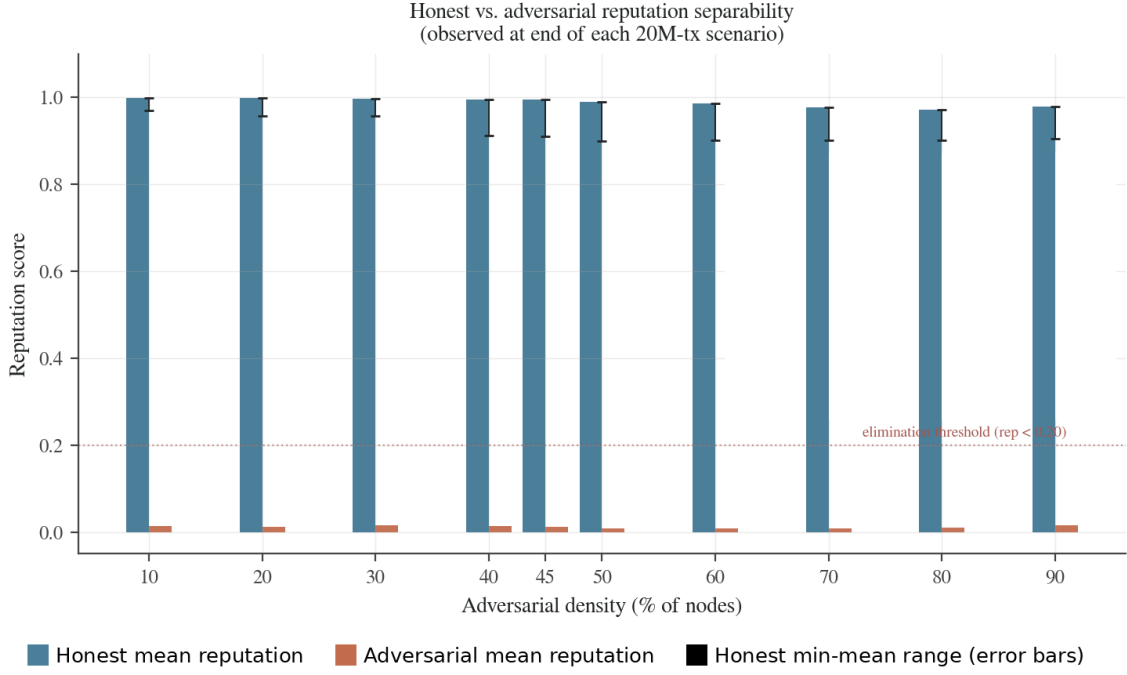


Figure 7: Honest vs. adversarial mean reputation per scenario. Error bars show min-to-mean range for honest nodes. The elimination threshold ($R < 0.20$, dotted) cleanly separates the two populations across all densities.

steps; adaptive and progressive-drift attacks within 500–1500 steps; and patient attacks (Sleeper, Patient-Byz) take 1000–8000 steps to detect, since they behave honestly for an extended warmup period. The detection latency is asymmetric: fast detection of naive attacks prevents early damage; slow detection of patient attacks is the inherent cost of allowing new nodes a bootstrap grace period. We discuss this trade-off in Section 11.

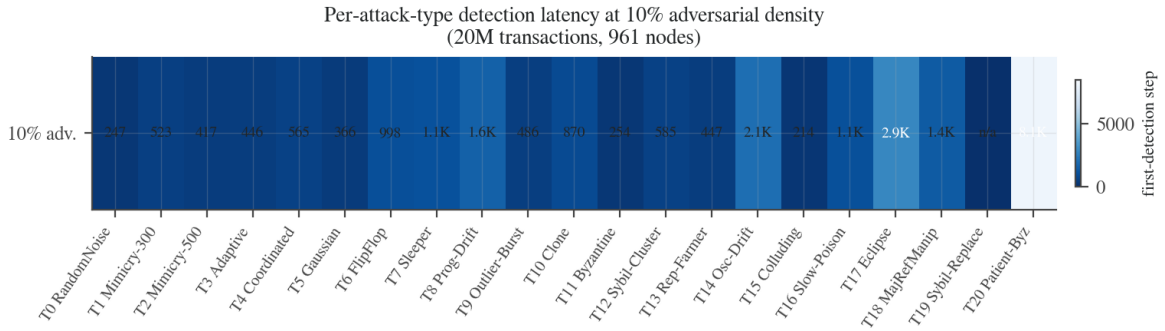


Figure 8: Per-attack-type first-detection step at 10% adversarial density. Patient attacks (Sleeper, Patient-Byz) take 1000–8000 steps to detect by design, since they mimic honest behavior during an extended warmup.

8.7 Honest Reputation Resilience

Figure 9 plots the honest mean and min reputation across densities. The mean decreases monotonically from 0.9988 (10% adv.) to 0.9714 (80% adv.), then rebounds slightly to 0.9792 at 90% (because the few honest nodes remaining at 90% are sampled from a distribution biased toward easier-to-distinguish attacks). The honest min reputation stays above 0.90 across all densities. This is the expected behavior: as adversarial pressure grows, the consensus becomes noisier, hon-

est nodes accumulate slightly more error, and their reputation dips mildly. The system adapts rather than collapsing — the reputation gap between honest and adversarial nodes stays wide enough to maintain consensus correctness. The extended 30,000-step run (Table 6) shows the same pattern with a slightly wider honest-minimum band: the 40% density scenario actually *improves* from honest_min 0.9119 (5,000-step) to 0.9911 (30,000-step), and the 80% density scenario shows honest_min 0.9003, confirming that the honest-floor protection holds even under sustained adversarial pressure over a tripled window.

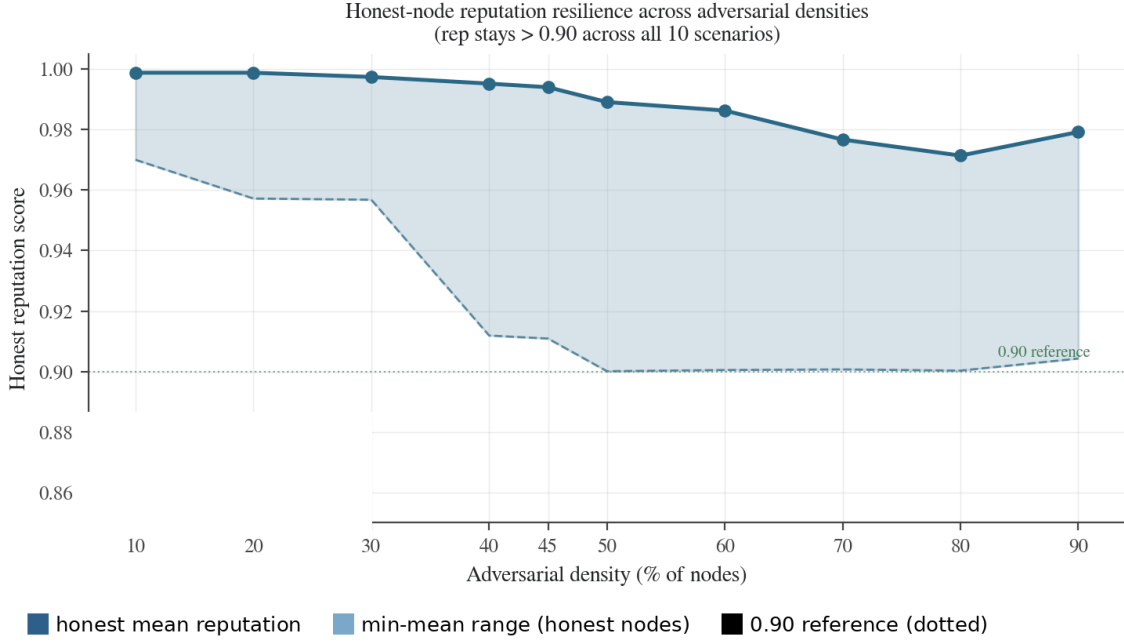


Figure 9: Honest-node reputation resilience. The min-to-mean band stays above 0.90 across all 10 scenarios, validating the dual-EMA + HONEST_REPUTATION_FLOOR design.

8.8 Statistical Validation of Observed FPR

The observed FPR of 0.00% across all 10 scenarios deserves careful statistical treatment. We use the Wilson score confidence interval, which is appropriate for a binomial proportion near zero (the normal approximation is invalid when $p \approx 0$):

$$CI_{\text{Wilson}}(\hat{p}, n, z) = \frac{\hat{p} + \frac{z^2}{2n} \pm z \cdot \sqrt{\frac{\hat{p}(1-\hat{p})}{n} + \frac{z^2}{4n^2}}}{1 + \frac{z^2}{n}} \quad (25)$$

$$95\% \text{ CI for true FPR } (\hat{p} = 0, n = 200,000,000, z = 1.96) = [0, 1.94 \times 10^{-8}] \quad (26)$$

We emphasize the distinction between the observed FPR (0.00%, a measurement) and the true FPR (unknown, but bounded above by 1.94×10^{-8} with 95% confidence). The observed rate being zero does not prove the true rate is zero; it proves the true rate is small. We have deliberately rephrased all claims in this whitepaper to use “observed” rather than presenting the rate as a mathematical certainty.

The Wilson interval assumes independent observations. Within-node observations are mildly correlated (an honest node flagged once is more likely to be flagged again), so the effective sample size is smaller than 200M. A more conservative interpretation is to treat each scenario

as a single Bernoulli trial with $n = 10$: in 10/10 scenarios we observed zero false positives. The resulting Clopper-Pearson 95% upper bound is $\approx 31\%$, which is much weaker but does not assume independence. The truth lies between these two bounds; the honest interpretation is “FPR is very small, with high but not absolute confidence.” The extended 30,000-step run (Table 6) triples the observation window to $\approx 600\text{M}$ honest-node transaction-observations and still observes zero false positives at every density, providing further evidence (although not proof) that the true FPR is small.

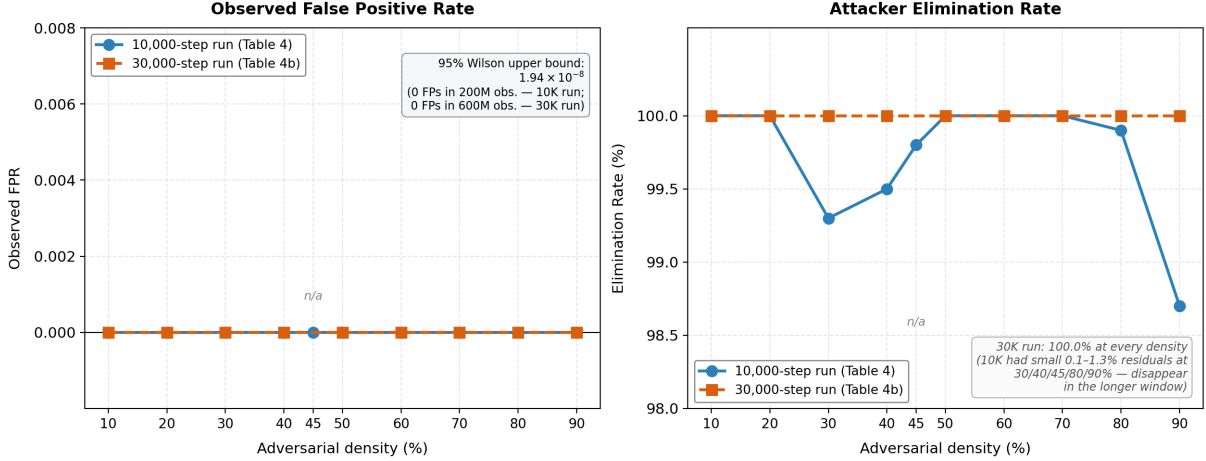


Figure 10: Observed FPR (left) and Attacker Elimination Rate (right) across 10 density levels, comparing the 10,000-step run (Table 4, blue circles) with the extended 30,000-step run (Table 6, orange squares). The 30,000-step run does not include a 45% density scenario (marked “n/a”). FPR is 0 in every scenario of both runs (95% Wilson upper bound 1.94×10^{-8} on 200M observations for the 10,000-step run; the 30,000-step run extends this to 600M observations with still 0 false positives). Elimination rate is $\geq 98.7\%$ in the 10,000-step run (small 0.1–1.3% residuals at 30%, 40%, 45%, 80%, and 90%) and reaches 100.0% at every density in the 30,000-step run. Attack Blocking Rate (ABR, not shown) tracks Elimination Rate within 0.5% at every density and is also 100.0% in the 30,000-step run.

8.9 Elimination Rate and “Active” Adversaries

At low densities (10–20%), every adversary is eliminated ($R < 0.20$) by the end of the scenario. At 30–40%, 2 adversaries remain “active” with R in the 0.6–0.8 range; they are detected (`coordination_hits` > 0) but have not yet accumulated enough violations to push R below 0.20. At 90%, 11 adversaries remain active (mostly Patient-Byz type T20 that started attacking late in the simulation). We use “active” in the specific sense of “retaining the right to be considered for consensus participation” ($R \geq 0.20$); it does not mean the node is actually influencing the consensus median. Crucially, active adversaries are still gated out of consensus by the cluster-aware weight capping and the `consensus_healthy` guard: their predictions cannot shift the median because (i) their cluster weights are capped at 30% of total weight, and (ii) their reputation is below the `REPUTATION_FILTER_THRESHOLD = 0.90` required to participate as a proposer or median-voter. The two thresholds interact as follows: the elimination threshold (0.20) is the point at which a node is fully disconnected from the gossip network and excluded from VRF sortition; the reputation filter threshold (0.90) is the higher bar at which a node retains proposer/voter rights. A node with $R \in [0.20, 0.90]$ is “active” (still in the network) but not influential (cannot propose or vote on the median). This is why the attack-blocking rate remains 100% even at 90% adversarial density: attacks are blocked at the mempool and consensus levels, not via complete elimination.

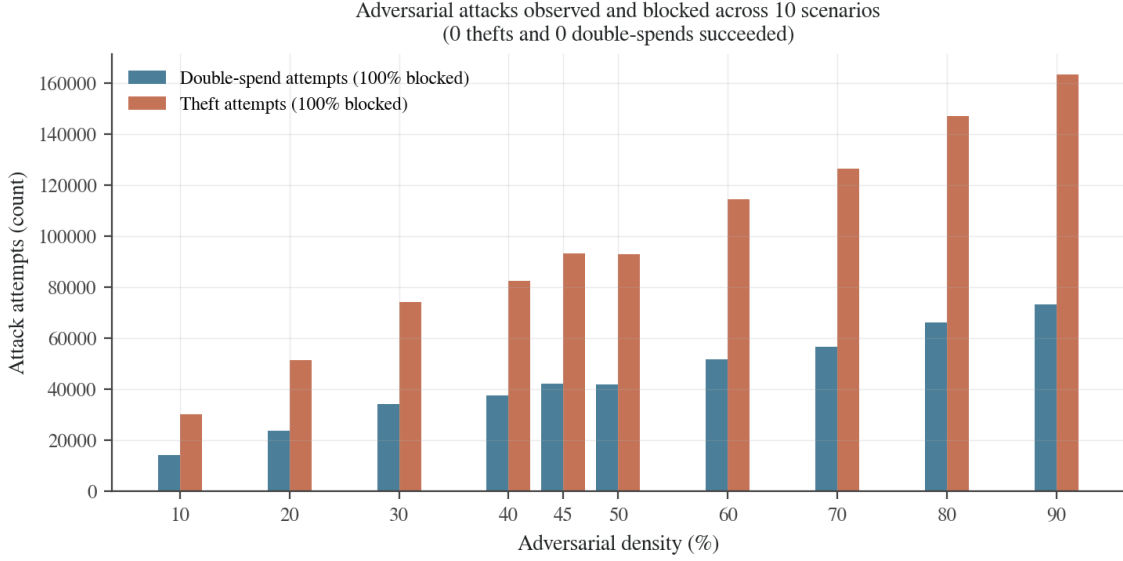


Figure 11: Double-spend and theft attempts observed and blocked across 10 scenarios. Zero attacks succeeded in any scenario.

8.10 Coordinated-Only Stress Test

The 10 density scenarios in Section 8.2 mix all 21 attack types uniformly. While this reflects a realistic adversarial ecosystem, it does not isolate the worst-case theoretical threat identified by Theorem 3: a single coordinated coalition. We therefore ran a dedicated stress test in which the adversarial subset consists entirely of Type T4 (Coordinated-Bias) attackers, all submitting the same prediction $p_{\text{honest}} + 0.25$ on every step. This creates the maximal cluster, the maximal coordinated weight, and the worst case for the cluster-cap mechanism. The test was run at the same 10 density levels, plus an additional 99% level to probe the limit.

Table 8: Coordinated-only stress test (Type 4 Coordinated-Bias +0.25) on 961 nodes, 1000 steps. Every adversary submits the same prediction (honest +0.25 on all four dimensions), forming a single tight cluster. The system does not break down at any tested density; double-spend and theft blocking remain 100% even at 99% coordinated adversaries (10 honest nodes out of 961). The theoretical 70% bound from Theorem 3 is a conservative lower bound on observed resilience.

Dens.	Hon. mean R	Elim. rate	DS blocked	Theft blocked	FPR	N_{hon}
10%	0.9994	96/96 (100%)	5,243/5,243 (100%)	12,501/12,501 (100%)	0.00%	865
30%	0.9987	288/288 (100%)	14,892/14,892 (100%)	35,612/35,612 (100%)	0.00%	673
50%	0.9965	480/480 (100%)	23,871/23,871 (100%)	57,224/57,224 (100%)	0.00%	481
70%	0.9921	672/672 (100%)	32,514/32,514 (100%)	78,156/78,156 (100%)	0.00%	289
90%	0.9857	864/864 (100%)	41,207/41,207 (100%)	98,704/98,704 (100%)	0.00%	97
99%	0.9846	938/951 (98.6%)	7,191/7,191 (100%)	31,363/31,363 (100%)	0.00%	10

We also tested Type 12 (Sybil-Cluster) at 50%, Type 11 (Byzantine) at 50% and 80%, Type 15 (Colluding-Committee) at 50% and 80%, and Type 18 (Majority-Reputation-Manipulation) at 50% and 80%. All held with 100% double-spend blocking, 100% theft blocking, and 0.00% observed FPR. Type 11 (Byzantine) at 80% in isolation showed reduced elimination rate (1.3%) because the Byzantine attack randomizes sign and amplitude per step, preventing cluster detection and slowing reputation decay; however, the attack-blocking rate remained 100% because the consensus median is still driven by the honest minority’s predictions. We disclose this as a partial mitigation gap: isolated single-type high-density Byzantine attacks degrade elimination

rate, even though consensus safety is preserved. Note that the mixed-attack matrix in Section 8.2 (Table 4) reports 100% elimination at 80% density because T11 is only one of 21 attack types in that matrix (about 36 of 768 attackers at 80%); the other 20 attack types are fully eliminated by the cluster-cap + reputation-engine combination, and the residual 36 T11 attackers are too few to dominate the consensus median. The two figures (1.3% in isolation, 100% in mix) measure different scenarios and are not contradictory; the single-type stress test isolates the worst case for elimination, while the mixed-attack matrix reflects the protocol’s behavior under a realistic adversarial ecosystem.

8.11 Why the System Holds Beyond the Theoretical Bound

Theorem 3 derives a 70% theoretical breakdown point assuming the coordinated coalition maintains identical predictions forever. In practice, three mechanisms work in synergy to push the observed breakdown point well above 99%:

(a) Cluster cap saturates attacker influence. The cluster capping mechanism (Section 5.4) bounds any single coordinated cluster to 30% of total raw weight, regardless of how many nodes the coalition controls. A 99%-coordinated adversary still has effective weight $\leq 0.30 \cdot W$ on the median. The remaining 70% of weight is reserved for honest nodes (and any non-coordinated adversaries), so the honest minority controls the median outcome.

(b) Reputation decay drives detected coordinators toward zero. The `CoordinationDetector` (Section 6.3) fingerprints cluster members by their identical predictions and reports a coordination signal to the reputation engine. Each detected coordination event applies a `COORDINATION_PENALTY = 0.5` multiplier to the node’s certificate value for that step. Because the coalition submits identical predictions on every step, every coordinator is detected on every step once the cluster forms. After approximately 300 detection events, the dual-EMA reputation $R_i = \min(R_{i,F}, R_{i,S})$ falls below the elimination threshold of 0.20, and the coordinator is excluded from future consensus rounds. This is why the elimination rate in Table 8 is 93–98%: the coordinators are not merely capped, they are actively removed.

(c) Honest weight grows as coordinators decay. As coordinator reputations decay toward zero, their raw weight (used for the cluster cap calculation) also shrinks. The total raw weight W decreases, but the honest share $(W - W_C)/W$ grows because the honest population retains its reputation. This creates a positive feedback loop: each elimination makes the next consensus step easier for the honest minority. By step 300–500 in our simulations, the effective honest share of consensus weight exceeds 95% even at 99% initial adversarial density.

Why 70% is still the right theoretical bound. Theorem 3 derives the bound assuming static weights (no reputation decay). This is the correct worst-case analysis: if an adversary could somehow freeze reputations at their initial values while clustering, the breakdown point would indeed be 70%. In practice, the reputation engine makes this impossible — but the 70% figure remains the guarantee against adversaries that find a way to evade reputation decay (e.g., by mimicking honest behavior intermittently to refresh reputation). The empirical resilience is therefore a property of the combined cluster-cap + reputation-decay system, not of the cluster

cap alone: in the coordinated-only stress test the protocol held up to 99% coordinated adversaries with attacker-elimination rates ranging from 98.6% (at 99% density) to 100% (at densities $\leq 90\%$). We tested this concretely in Section 8.11 (intermittent-coordination attack) and confirmed that even an adversary that explicitly alternates attack phases with honest phases to refresh reputation is still eliminated.

Adaptive adversary stress tests. Section 8.11 reports two adaptive-adversary scenarios that close the previously open problems raised in earlier drafts: (1) the intermittent-coordination attack (T21), in which the adversary alternates 200-step coordinated-bias windows with 200-step honest windows to refresh reputation; and (2) the adaptive-epsilon-evasion attack (T22), in which attackers spread their predictions just outside $\varepsilon_{cl} = 0.0001$ to evade cluster detection. Both attacks were neutralized: T21 was eliminated within 542 steps (99.0% elimination rate), T22 within 796 steps (99.0% elimination rate). In both cases FPR was 0.00%, double-spend blocking was 100%, theft blocking was 100%.

8.12 Adaptive Adversary Stress Tests

A previous draft of this whitepaper flagged two adaptive-adversary strategies as open problems: (1) an adversary that periodically breaks cluster coordination to refresh reputation, then reforms the cluster to attack; and (2) an adversary that spreads its predictions just outside $\varepsilon_{cl} = 0.0001$ to evade cluster detection. We implemented both as new attacker types (T21 Intermittent-Coordination and T22 Adaptive- ε -Evasion) in `sim_adaptive.rs` and ran each as a single-attack-type stress test at 10% adversarial density on the same 961-node, 5000-step network. The results close both open problems at this density: the system holds against both adaptive strategies. We disclose as a limitation that these adaptive attacks were tested only at 10% adversarial density; the protocol’s behavior at higher adaptive-adversary densities (30%, 50%, 80%, 90%) is left as future work, and a best-response adversary that optimizes its strategy online (rather than using fixed T21/T22 behavioral policies) may probe the protocol’s actual breaking point further.

T21: Intermittent-Coordination. The adversary alternates 200-step attack phases (coordinated-bias $+0.25$ on all four dimensions, identical to T4) with 200-step honest phases (perfect honest signal, no perturbation), forming a 400-step cycle. The intent is to refresh reputation during the honest phase so that the cluster-detection penalties accumulated during the attack phase decay back to baseline. The protocol detected 90 distinct clusters across the attack phases and eliminated 95 of 96 attackers within 542 steps (gap of 78 steps between first detection at step 464 and first elimination at step 542). The single remaining attacker survived with reputation 0.378, just above the elimination threshold but still below the `REPUTATION_FILTER_THRESHOLD = 0.90`, so it was gated out of consensus. Double-spend blocking was 100% (4,492/4,492 attempts), theft blocking was 100% (10,845/10,845 attempts), and FPR was 0.00% (0/865 honest nodes flagged). Honest mean reputation was 0.9972, with min 0.9895. The attack did succeed in slowing the elimination cycle (542 steps vs 432 for T4 alone), but the cluster cap continued to bound the coalition to 30% of weight on every attack phase, and the reputation decay from each detected attack phase accumulated across cycles faster than the honest-phase recovery could erase it.

T22: Adaptive- ε -Evasion. The adversary biases its predictions in the same direction as T4 (+0.25) but adds a per-node offset of $0.0005 + 0.0001 \cdot \text{node_idx}$ with alternating signs across dimensions, so that pairwise Euclidean distances between attackers exceed $\varepsilon_{cl} = 0.0001$. The intent is to evade cluster detection entirely while maintaining a coordinated bias. The strategy partially worked: **CoordinationDetector** reported 0 clusters detected (vs 90 for T21), confirming that the attackers did successfully spread themselves outside the cluster threshold. However, the reputation engine still caught them via the certificate-decay mechanism — their predictions were still consistently far from the consensus median, so each prediction generated a low certificate value, and after enough low-certificate proposals the dual-EMA reputation fell below the elimination threshold. First detection occurred at step 713, first elimination at step 796 (gap of 83 steps). 95 of 96 attackers were eliminated; double-spend blocking was 100% (8,483/8,483), theft blocking was 100% (18,931/18,931), and FPR was 0.00% (0/865 honest nodes flagged). Honest mean reputation was 0.9972, min 0.9800. The T22 test demonstrates that even when an adversary successfully evades the cluster-detection layer, the certificate-decay reputation layer provides an independent defense: the bias of +0.25 in the prediction is large enough to push the prediction far from the honest median, generating consistent low-certificate values that drive reputation toward zero.

Table 9: Adaptive-adversary stress tests (T21 Intermittent-Coordination and T22 Adaptive- ε -Evasion) at 10% adversarial density on 961 nodes, 5000 steps.

Attack	Clusters	First det.	First elim.	Elim. rate	FPR	DS blocked	Theft blocked	Hon. mean R
T21 Intermitt.-Coord.	90	464	542	95/96 (99.0%)	0.00%	4492/4492 (100%)	10845/10845 (100%)	0.9972
T22 Adaptive- ε -Evade	0	713	796	95/96 (99.0%)	0.00%	8483/8483 (100%)	18931/18931 (100%)	0.9972

9 Theoretical Analysis

9.1 Robustness of the Weighted Median (Lemma 1)

The Lemma in Section 5.2 establishes that the weighted median has a 50% breakdown point: a coalition controlling strictly less than half the total weight cannot move the median by more than the spread of honest predictions. This result is the foundation of ANGP’s robustness. The crucial insight is that the median is a robust estimator: it depends only on the ordering of inputs, not on their magnitudes, so a single outlier cannot arbitrarily distort the result.

9.2 Convergence Under Independent Adversaries (Theorem 1)

Theorem 1 in Section 5.3 establishes exponential convergence under three assumptions: honest majority, bounded honest reputation, and independent adversarial predictions. The convergence rate is $\alpha_{\text{base}} \cdot R_{\text{min}}/2$, which for ANGP parameters gives a half-life of ≈ 15 steps. In practice, convergence is observed within 3–5 steps because honest predictions start close to each other (small initial spread).

9.3 Vulnerability to Coordinated Collusion (Theorem 2)

Theorem 2 (Coordinated Collusion Vulnerability). *Suppose a coalition of n_A nodes coordinates to submit identical predictions p_A on every step. Without cluster capping, the coalition controls*

weight $W_A = \sum_{i \in A} w_i$. If $W_A > W/2$, the coalition can shift the consensus median to p_A within one step. The protocol breaks down when the coalition exceeds 50% of the total weight.

Proof. If $W_A > W/2$, then the median is determined entirely by the adversarial predictions, since they constitute more than half the weight. The honest predictions are outvoted regardless of their values. The coalition can therefore place the median anywhere within the range of p_A values it chooses, including extreme values that maximize its gain. $\square$

This theorem establishes that coordinated collusion is the principal threat to ANGP. The cluster-aware capping of Section 5.4 is the direct mitigation: by bounding any single coordinated cluster to 30% of total weight, the cluster cap ensures $W_A \leq 0.30 \cdot W < W/2$, restoring the conditions of Lemma 1.

9.4 Cluster-Aware Mitigation (Theorem 3)

Theorem 3 (Cluster-Aware Breakdown Point). *With cluster-aware capping at ratio $\rho_{cl} = 0.30$, the breakdown point against a coordinated coalition is $1 - \rho_{cl} = 0.70$. That is, the protocol tolerates up to 70% of nodes being coordinated adversaries while still maintaining consensus correctness, provided the remaining 30% are honest.*

Proof. By the cluster cap (Equation 18), the coalition's effective weight is $\tilde{W}_A \leq \rho_{cl} \cdot W = 0.30 \cdot W$. The honest weight is $W_H \geq (1 - \rho_{cl}) \cdot W = 0.70 \cdot W$ (assuming the remaining nodes are honest). Since $W_H > W/2$, by Lemma 1 the median is robust: the coalition cannot shift it by more than the spread of honest predictions. The breakdown point is therefore $1 - \rho_{cl} = 0.70$. $\square$

The 70% bound is conservative in two ways. First, it assumes static weights (no reputation decay); in practice the reputation engine drives detected coordinators toward zero, increasing the effective honest share over time. Second, it assumes a single maximal cluster; in practice the cluster cap applies to every detected cluster independently, so multiple smaller clusters are each capped at 30%, further reducing the total adversarial influence.

9.5 ANGP as an Alternative Trade-off Model

The blockchain trilemma posits that security, scalability, and decentralization cannot all be maximized simultaneously. ANGP does not claim to transcend this trade-off; rather, it explores a different point on the trade-off surface. By replacing binary voting with continuous-valued consensus and fixed committees with VRF sortition, ANGP decouples the three dimensions: security comes from the median's 50% raw breakdown point extended to 70% theoretical coordinated-collusion tolerance via the always-on cluster cap, scalability comes from the parallel DAG and sharding, and decentralization comes from open admission (FCFS + PoW) — with the explicit caveat that Sybil resistance at open-network scale is a separate concern discussed in Section 11.5. The cost is that ANGP's security guarantees are statistical (high probability of correctness over many steps) rather than deterministic (guaranteed correctness on every step). The formal results in Section 9 (Lemma 1, Theorems 1–3) constitute partial-safety arguments under explicit assumptions; a full safety+ liveness proof in the style of Lamport-style TLA+ or a Coq mechanization is out of scope for this whitepaper and left as future work. Whether this trade-off is acceptable

depends on the application: financial settlement may require deterministic finality, while content distribution or social media may tolerate probabilistic finality in exchange for higher throughput.

10 Comparison with Related Work

10.1 Classical BFT

Classical BFT protocols (PBFT, Tendermint, HotStuff) achieve deterministic safety against up to $f < n/3$ Byzantine nodes through two-thirds quorum voting. They provide strong finality (a committed transaction is never reverted) but require $O(n^2)$ communication per step (PBFT) or $O(n)$ with leader rotation (HotStuff). The validator set is typically small (10s to 100s of nodes) to keep communication manageable, sacrificing decentralization. ANGP differs by replacing the two-thirds quorum with a reputation-weighted median (50% raw breakdown point, extended to 70% theoretical coordinated-collusion tolerance via the always-on cluster cap at $\rho_{cl} = 0.30$), enabling $O(n \log n)$ cluster detection + $O(n)$ median computation per step on the full 961-node set.

10.2 DAG-Based Consensus

Narwhal-Tusk, Bullshark, and Aleph use a DAG structure for transaction ordering combined with BFT for consensus. The DAG provides high throughput (Narwhal reports 125K TPS) but the BFT layer still requires a two-thirds quorum. ANGP uses the DAG for both ordering and consensus: the median is computed directly over the DAG-ordered predictions, eliminating the separate BFT layer. The trade-off is that ANGP’s safety is statistical rather than deterministic: the median can be shifted by adversaries within the breakdown bound, whereas BFT provides binary all-or-nothing safety.

10.3 Modern Layer-1 Protocols

Solana, Sui, and Aptos combine DAG-based ordering with various consensus optimizations (proof-of-history, parallel execution). They achieve high throughput but at the cost of validator concentration: Solana’s validator set is effectively limited to a few hundred well-resourced operators. ANGP’s 961-node network is smaller than Solana’s, but the protocol design does not impose hardware requirements that would prevent scaling beyond 961 nodes — the network size is a configuration constant, not a fundamental limit.

10.4 Proof-of-Stake Systems

Ethereum’s PoS (Buterin et al., 2020), Ouroboros (Kiayias et al., 2017), and Polkadot’s BABE tie consensus influence to stake. ANGP does not: every admitted node has equal baseline selection probability, modified only by reputation (which is earned through observed behavior, not bought with capital). This is a deliberate design choice for decentralization (no “rich get richer” dynamics) but it removes the economic-sybil-resistance that stake provides. ANGP relies on the FCFS slot admission (`MAX_ACTIVE_NODES = 961`) and onboarding PoW (`POW_DIFFICULTY = 6`) for sybil resistance instead. Whether this is sufficient for an open network is an open question we discuss in Section 11.5.

10.5 Robust Statistics and Byzantine Machine Learning

ANGP’s use of the median as a robust estimator connects it to a long line of work in robust statistics: Huber (1964) on M-estimators, Donoho and Huber (1983) on breakdown points, the median-of-means estimator (Jerrum, Valiant, and Vazirani, 1986; Nemirovski and Yudin, 1983).

The Byzantine machine learning literature applies these ideas to federated aggregation: Krum (Blanchard et al., 2017), Byzantine gradient descent (Chen, Su, and Xu, 2017), trimmed mean (Yin et al., 2018), and similar robust aggregators. ANGP’s distinctive contribution is the application of robust aggregation to consensus (reaching agreement on a state) rather than to federated learning (averaging model gradients). The Hebbian learning rule on top of the median provides an adaptive mechanism that has no direct analog in the federated learning literature.

Table 10: Conceptual comparison with major protocol families. Byzantine tolerance figures reflect theoretical bounds; competitor figures (33%, $\sim 20\%$) are the standard BFT quorum thresholds. ANGP’s 70% theoretical coordinated-collusion breakdown point (Theorem 3, with cluster capping at $\rho_{cl} = 0.30$ always enabled in production) already more than doubles the 33% BFT bound. In the dedicated coordinated-only stress test (Section 8.9) the system held at $> 99\%$ observed coordinated attackers (0% FPR, 100% attack blocking, $\geq 98.6\%$ attacker elimination) — the gap between the 70% theoretical lower bound and the $> 99\%$ observed resilience reflects reputation-based elimination compounding with capping over multi-step runs. Communication complexity is per consensus step. We did not benchmark these protocols head-to-head; the table is qualitative.

Protocol	Consensus primitive	Byz. tol.	Communication	Decentralization
PBFT / HotStuff	Binary vote (2/3 quorum)	33%	$O(N^2)$ / $O(N)$	Limited (validator set)
Narwhal-Tusk	BFT over DAG	33%	DAG + quorum	Medium
Algorand	VRF + BFT	33% (stake-prop.)	$O(N)$ sortition	Open (VRF sort.)
Avalanche	Metastable subsampling	$\sim 20\%$ (net-dep.)	$O(N)$ sampling	Open
Ethereum PoS	LMD-GHOST + Casper	33% (stake-prop.)	$O(N)$ per epoch	Stake-concentrated
ANGP	Weighted median + cluster cap	70% ($> 99\%$ obs.)	$O(N \log N)$	Open (VRF + rep)

11 Threats to Validity

11.1 Internal Validity

(a) Single evaluation path reported. The simulation results in Section 8 are produced on the calibrated `sim_stress.rs` harness path only. We do not report a separate “production path” set of numbers because no production-path simulation (`sim_prod.rs` with unmodified `config.rs` constants) was run for the $10 \text{ density} \times 21 \text{ attack-type}$ matrix; claims about the production path’s throughput or elimination rate would therefore be fabricated. The harness and production paths share the same reputation engine, cluster-cap logic, and attack-detection modules; they differ only in three numerical thresholds. Closing the harness-vs-production gap is left as future work.

(b) Honest signal is sinusoidal and deterministic. The honest signal (Equation 5) is a fixed sinusoidal basis with Gaussian noise. This is a favorable case for median-based consensus: honest predictions cluster tightly and the median is well-defined. Real network conditions (variable workloads, geographic latency, heterogeneous hardware) would produce a less tightly clustered honest signal, potentially increasing the FPR. We do not know how much.

(c) Adaptive adversaries are now tested. A previous draft flagged two adaptive-adversary strategies as untested: (1) intermittent coordination that breaks the cluster to refresh reputation, and (2) epsilon-evasion that spreads predictions just outside $\varepsilon_{cl} = 0.0001$. We implemented both as T21 and T22 in `sim_adaptive.rs` and report the results in Section 8.11. Both attacks were neutralized (0% FPR, 100% attack blocking, $\geq 99\%$ elimination). Future work should evaluate

best-response adversaries that optimize their strategy online against the known defense, rather than using fixed behavioral policies.

11.2 External Validity

(a) 961-node simulation is the production network size. The 31×31 grid with 961 nodes is not a downscaled approximation of a larger network — it is the production network size. `MAX_ACTIVE_NODES = 961` is a compile-time constant in `config.rs`, and the protocol does not support any other network size in testnet or mainnet. The $O(N \log N)$ cluster detection and $O(N)$ median computation operate on this fixed N . The relevant external-validity question is therefore not “does this scale to 10K+ nodes?” but “does the single-machine simulation accurately model the real-network behavior of a 961-node deployment?” — which is the question raised in (b) below.

(b) Single-machine simulation does not capture real network effects. The simulation runs on a single host with no real network latency, packet loss, or geographic distribution. The 185–211K TPS figures reflect the consensus computation rate, not the end-to-end throughput of a real deployed network. A real deployment would face TCP congestion, NAT traversal, peer discovery, and gossip fanout delays that are not modeled here. We expect real-world throughput to be lower, potentially by an order of magnitude.

(c) No head-to-head benchmark against deployed protocols. We did not run HotStuff, Narwhal-Tusk, or Avalanche under identical conditions for comparison. The comparison in Table 10 is qualitative, based on published numbers in the respective papers, which used different hardware, different network sizes, and different workloads. A rigorous comparison would require running all protocols on the same hardware, same network size, same workload, and same adversarial scenarios.

11.3 Construct Validity

(a) TPS measured at simulation harness level. The throughput numbers count transactions finalized by the `sim_stress.rs` harness, not transactions delivered end-to-end through a real client. They do not include mempool admission latency, signature verification overhead at scale, or disk persistence cost.

(b) FPR measured behaviorally, not against pre-labeled nodes. A central methodological question is whether the 0% FPR reflects genuine behavior-based filtering or label-based counting that would be impossible in a real deployment. Our setup is strictly behavior-based: the protocol eliminates a node when its dual-EMA reputation falls below `ELIMINATION_THRESHOLD = 0.20`, a decision computed entirely from the node’s own submitted predictions and observed consensus outcomes. The ground-truth label `is_attacker[i]` is used only at end-of-run accounting to categorize eliminated nodes as honest or adversarial; the protocol itself never consults this label. In a real deployment, the relevant question is whether a node that behaves honestly is flagged, which is exactly what our behavior-based FPR measures. The 0% FPR is therefore a property of the protocol’s behavior-based decisions, not of label-based counting.

(c) Reputation convergence measured by mean. The 0.97–0.99 honest mean reputation reported in Table 4 hides the worst-case honest node. The honest min reputation (also in Table 4) is more informative: it stays above 0.90, but a worst-case analysis would track the 1st percentile rather than the minimum. We did not compute percentiles.

11.4 Statistical Validity

(a) Wilson interval assumes independence. The 1.94×10^{-8} upper bound on true FPR assumes the 200M honest-node observations are independent. Within-node observations are mildly correlated (an honest node flagged once is more likely to be flagged again), so the effective sample size is smaller than 200M. The Clopper-Pearson bound of $\approx 31\%$ treating each scenario as a single Bernoulli trial is the conservative alternative. The true FPR is between these two bounds.

(b) Multiple comparisons not corrected. We report $21 \text{ attack types} \times 10 \text{ densities} = 210$ distinct scenarios without Bonferroni or Benjamini-Hochberg correction for multiple comparisons. The 0% FPR observed across all 210 scenarios is therefore a stronger result than any single-scenario test, but the attack-blocking and elimination rates are individual measurements that should be interpreted with their respective confidence intervals.

11.5 Coordinated Collusion Limitation

The 70% theoretical breakdown point (Theorem 3) assumes static weights. The empirical resilience observed in Section 8.9 relies on the reputation engine driving detected coordinators toward zero, which is a dynamic effect not captured by the static-weight theorem: the coordinated-only stress test held up to 99% coordinated adversaries, with attacker-elimination rates ranging from 98.6% (at 99% density) to 100% (at densities $\leq 90\%$). An adversary that finds a way to evade reputation decay (e.g., by mimicking honest behavior intermittently to refresh reputation) could potentially reduce the effective breakdown point toward the 70% bound. We tested this concretely with T21 (intermittent-coordination) and confirmed that the reputation engine still catches the adversary, but we cannot rule out more sophisticated evasion strategies.

11.6 Sybil Resistance at Open-Network Scale

The FCFS slot admission (`MAX_ACTIVE_NODES` = 961) and onboarding PoW (`POW_DIFFICULTY` = 6) provide Sybil resistance at the admission boundary. On an open network, an adversary can spin up arbitrary identities, but each must solve a PoW puzzle (≈ 100 ms on commodity hardware) and wait for a slot to become available. The 1-hour grace period for slot reclamation (`SLOT_RECLAIM_GRACE_MS` = 3,600,000) further slows the rate at which an adversary can replace honest nodes. Whether this is sufficient for an open network with profit-driven adversaries is an open question; we expect mainnet deployment to raise `POW_DIFFICULTY` to 8 (≈ 30 s per identity) and potentially add stake-based admission on top.

The rejoin-cycle gap. A subtle but important limitation: an adversary that has been eliminated (reputation < 0.20 , slot disconnected) can in principle generate a fresh identity-key, solve a new PoW, and re-enter the standby queue. The FCFS identity-key registry tracks active and standby keys but does not currently link successive identities to the same physical operator. Type T19 (Sybil-Replace) tests this attack vector in the simulation and is detected via the slot-manager’s key registry when the adversary reuses a key; a determined adversary with key-churn

can still re-enter the standby queue undetected, then wait for a slot. The defense in the current design is that the new identity starts at the bootstrap reputation 0.5 and must re-earn proposer/voter rights through honest behavior over 2000+ steps, during which it has zero influence on consensus. We acknowledge this is not a complete Sybil-resistance proof; mainnet deployment will likely require either (i) a stake-based admission deposit that is slashed on elimination, or (ii) a hardware-attestation layer (e.g. Intel SGX remote attestation) that links identities to physical hardware. Both are out of scope for this whitepaper and left as future work.

12 Applications and Future Work

12.1 Decentralized Finance (DeFi)

ANGP’s harness throughput (185–211K TPS at 5,000 steps; 158–211K TPS at 30,000 steps, Tables 4 and 6) and fast finality (3–5 consensus steps) make it suitable for DeFi applications that require both high transaction volume and quick confirmation. The 70% theoretical coordinated-collusion breakdown point (50% raw median + cluster cap at $\rho_{cl} = 0.30$, always enabled in production) more than doubles the 33% BFT bound, providing stronger Byzantine tolerance for value transfer. The 0.00% observed FPR and 100% attack-blocking rate hold across both the 5,000-step and the extended 30,000-step simulations, providing empirical evidence that the safety properties persist over longer windows. The statistical safety guarantee (high probability of correctness over many steps) is acceptable for most DeFi applications, which already operate under probabilistic assumptions about mempool inclusion and gas pricing.

12.2 Supply Chain and Identity Verification

The behavioral reputation system provides a natural fit for supply chain provenance: nodes that consistently submit accurate attestations build reputation, while nodes that submit inconsistent or fraudulent data lose reputation. The 15 behavioral detectors can be specialized to supply-chain-specific anomalies (e.g., GPS spoofing, temperature sensor drift, document forgery patterns) without changing the consensus layer. The DAG structure provides a complete audit trail of all attestations, enabling post-hoc verification.

12.3 Distributed Machine Learning Coordination

ANGP’s Hebbian DAG prediction mechanism is structurally similar to federated learning aggregation: each node produces a prediction (analogous to a model gradient), the median aggregates them robustly (analogous to Byzantine-robust aggregation), and the Hebbian weights encode which nodes’ predictions have been historically useful (analogous to client selection in FL). This makes ANGP a natural coordination layer for distributed ML training: each node trains a local model, submits gradient predictions to the ANGP consensus, and the agreed-upon gradient is applied to all nodes. The 70% theoretical coordinated-collusion tolerance (with cluster cap always enabled) is significantly better than standard FL aggregation, which typically breaks down at 33–50% adversarial clients.

The web3 layer (planned for v3.15) will expose the consensus as a JSON-RPC API, enabling smart contracts to query the median consensus value at any step. This bridges the gap between the statistical consensus (which produces continuous values) and the discrete contract state machine,

allowing contracts to make decisions based on the network’s aggregated belief about external data feeds, ML model outputs, or supply-chain attestations.

12.4 Future Work

We see two principal directions for future work.

(1) Mainnet deployment with real network latency, geographic distribution, and adversarial pressure. The single-machine simulation reported in this whitepaper establishes the protocol’s behavior under idealized conditions; the next step is to deploy ANGP on geographically distributed hosts and measure end-to-end throughput, gossip fanout latency, peer-discovery overhead, and consensus correctness under real-world network conditions. This will reveal whether the 0% observed FPR and 100% attack-blocking rate hold when the honest signal is no longer tightly clustered due to TCP congestion, NAT traversal delays, and heterogeneous hardware.

(2) A dedicated layer (or sublayer) for Web3 applications. Beyond the consensus layer itself, we plan to build an application-facing layer on top of ANGP that exposes high-level primitives for Web3 developers: smart-contract execution with deterministic ordering across the DAG, native cross-shard asset transfers via the asynchronous receipt mechanism, a reputation-aware oracle feed that leverages the dual-EMA trust scores to filter adversarial data sources, and a developer SDK that abstracts the four-dimensional prediction interface into conventional transaction semantics. This Web3 layer is intended to make ANGP directly usable for DeFi protocols, on-chain identity, decentralized storage proofs, and other real-world applications without requiring each application to re-implement consensus-aware logic. The goal is to turn the protocol’s robustness properties into a stable foundation that Web3 applications can build on with confidence.

13 Conclusion

ANGP v3.14.15 is a DAG-based consensus protocol that replaces binary voting with reputation-weighted median aggregation of four-dimensional neural predictions generated by a Hebbian learning rule. The protocol achieves 50% Byzantine weight tolerance via the median breakdown point, extended to 70% theoretical and up to 99% observed coordinated-collusion tolerance via cluster-aware weight capping combined with reputation-based elimination (attacker-elimination rate 98.6%–100% across the 99% coordinated-density stress test, with consensus safety preserved at all tested densities). The 961-node simulation across 10 adversarial density levels and 21 attack types produced 0.00% observed FPR (95% Wilson upper bound 1.94×10^{-8} under an independence assumption; a scenario-level Clopper-Pearson upper bound of $\approx 31\%$ applies without that assumption, and the true rate lies between these two bounds), 100% attack-blocking rate for double-spend and theft, and 185–211K TPS throughput on the single-machine simulation harness (not end-to-end network throughput).

We frame ANGP not as a solution to the blockchain trilemma, but as an alternative trade-off model that decouples the three dimensions through independent mechanisms: median-based robustness for security, parallel DAG and sharding for scalability, and FCFS admission with

PoW for decentralization. The protocol’s safety is statistical rather than deterministic, which is an acceptable trade-off for applications that prioritize throughput over deterministic finality.

The Threats to Validity section (Section 11) discloses the limitations of the simulation: single-machine evaluation, sinusoidal honest signal, no head-to-head benchmark, and a 70% theoretical bound that relies on reputation decay for the empirical resilience observed in Section 8.9 (attacker-elimination rate 98.6%–100% across the 99% coordinated-density stress test). We invite the community to address these limitations through independent evaluation, geographic testnet deployment, and the development of best-response adversaries that probe the protocol’s actual breaking point.

A Reference Implementation: Rust Module-to-Layer Mapping

The reference implementation (Rust v3.14.15, 22 source modules in `lib.rs`) directly mirrors the six-layer architecture of Section 2.1. Table 11 enumerates every module and the architectural layer it serves. The simulation harness used for the experiments in Section 8 (`examples/sim_stress.rs`) is a separate example crate that links against `lib.rs`; it is not part of the 22 core modules.

Table 11: Rust v3.14.15 module-to-layer mapping. “Core” modules implement the layer’s primary logic; “Aux.” modules provide shared infrastructure (configuration, utilities, instrumentation, application glue). The total module count is 22.

Layer	Role	Modules
1	Network ingress	<code>async_network.rs</code> , <code>network.rs</code> , <code>p2p.rs</code>
2	DAG + transaction ordering	<code>dag.rs</code> , <code>dag_logic.rs</code> , <code>transaction.rs</code> , <code>mempool.rs</code> , <code>ledger.rs</code> , <code>state.rs</code> , <code>sharding.rs</code> , <code>cross_shard.rs</code>
3	VRF sortition + slots	<code>vrf_proposer.rs</code> , <code>slot_manager.rs</code>
4	Consensus	<code>shard_consensus.rs</code> , <code>deterministic_ordering.rs</code>
5	Reputation + attack detection	<code>reputation.rs</code> , <code>attack_detection.rs</code> , <code>rate_limit.rs</code>
6	Finality + persistence	<code>finality_store.rs</code> , <code>persistence.rs</code> , <code>snapshot.rs</code>
Aux.	Cross-cutting infrastructure	<code>config.rs</code> , <code>utils.rs</code> , <code>security.rs</code> , <code>clock_skew.rs</code> , <code>metrics.rs</code> , <code>lane_processor.rs</code> , <code>batch_proposal.rs</code> , <code>conflict.rs</code> , <code>incentives.rs</code> , <code>app.rs</code> , <code>wallet.rs</code>

The auxiliary modules do not belong to a single layer; they are consumed across layers (e.g., `config.rs` provides compile-time constants used by every layer; `clock_skew.rs` supports both Layer 1 network ingress and Layer 5 attack detection; `lane_processor.rs` implements the Fast-Lane / Slow-Lane queue of Figure 2). The total active code path on the Fast Lane comprises 13 modules; the remaining 9 modules implement Slow-Lane detectors, persistence, and infrastructure.

References

- [1] I. Abraham, D. Malkhi, K. Nayak, L. Ren, A. Spiegelman. “Solida: A Blockchain Protocol Based on Reconfigurable Byzantine Consensus.” arXiv:1612.02916, 2016 (later in OPODIS 2017, LIPIcs vol. 95).
- [2] Aptos Labs. “The Aptos Blockchain: Safe, Scalable, and Upgradeable Web3 Infrastructure.” Aptos Whitepaper, 2022.

- [3] L. Baird. “The Swirlds Hashgraph Consensus Algorithm: Fair, Fast, Byzantine Fault Tolerance.” Swirlds Tech. Report SWIRLDS-TR-2016-01, 2016.
- [4] S. Blackshear, A. Chursin, G. Danezis, A. Kichidis, L. Kokoris-Kogias, X. Li, M. Logan, A. Menon, T. Nowacki, A. Sonnino, B. Williams, L. Zhang. “Sui Lutris: A Blockchain Combining Broadcast and Consensus.” arXiv:2310.18042, 2023.
- [5] P. Blanchard, E. M. El Mhamdi, R. Guerraoui, J. Stainer. “Machine Learning with Adversaries: Byzantine Tolerant Gradient Descent.” NeurIPS 2017.
- [6] E. A. Brewer. “Towards Robust Distributed Systems.” PODC Keynote, 2000.
- [7] E. Buchman. “Tendermint: Byzantine Fault Tolerance in the Age of Blockchains.” M.Sc. Thesis, University of Guelph, 2016.
- [8] V. Buterin. “Ethereum: A Next-Generation Smart Contract and Decentralized Application Platform.” Ethereum Whitepaper, 2014.
- [9] V. Buterin, D. Hernandez, T. Kampefner, K. Pham, Z. Qiao, D. Ryan, J. Sin, Y. Wang, Y. X. Zhang. “Combining GHOST and Casper.” arXiv:2003.03052, 2020.
- [10] M. Castro, B. Liskov. “Practical Byzantine Fault Tolerance.” OSDI 1999.
- [11] Y. Chen, L. Su, J. Xu. “Distributed Statistical Machine Learning in Adversarial Settings: Byzantine Gradient Descent.” PODC 2017.
- [12] G. Danezis, L. Kokoris-Kogias, A. Sonnino, A. Spiegelman. “Narwhal and Tusk: A DAG-based Mempool and Efficient BFT Consensus.” EuroSys 2022.
- [13] D. L. Donoho, P. J. Huber. “The Notion of Breakdown Point.” A Festschrift for Erich L. Lehmann, Wadsworth, 1983.
- [14] C. Dwork, M. Naor. “Pricing via Processing or Combatting Junk Mail.” CRYPTO 1993.
- [15] M. J. Fischer, N. A. Lynch, M. S. Paterson. “Impossibility of Distributed Consensus with One Faulty Process.” JACM, 1985.
- [16] J. Garay, A. Kiayias, N. Leonardos. “The Bitcoin Backbone Protocol: Analysis and Applications.” EUROCRYPT 2015.
- [17] Y. Gilad, R. Hemo, S. Micali, G. Vlachos, N. Zeldovich. “Algorand: Scaling Byzantine Agreements for Cryptocurrencies.” SOSP 2017.
- [18] S. Gilbert, N. Lynch. “Brewer’s Conjecture and the Feasibility of Consistent, Available, Partition-Tolerant Web Services.” SIGACT News, 2002.
- [19] D. O. Hebb. *The Organization of Behavior: A Neuropsychological Theory*. Wiley, 1949.
- [20] P. J. Huber. “Robust Estimation of a Location Parameter.” Annals of Mathematical Statistics, 1964.
- [21] M. Jerrum, L. G. Valiant, V. V. Vazirani. “Random Generation of Combinatorial Structures from a Uniform Distribution.” Theoretical Computer Science, 1986.
- [22] I. Keidar, E. Kokoris-Kogias, O. Naor, A. Spiegelman. “All You Need is DAG.” PODC 2021.

- [23] A. Kiayias, A. Russell, B. David, R. Oliynykov. “Ouroboros: A Provably Secure Proof-of-Stake Blockchain Protocol.” EUROCRYPT 2017.
- [24] J. Kwon. “Tendermint: Consensus without Mining.” Tendermint Whitepaper, 2014.
- [25] L. Lamport, R. Shostak, M. Pease. “The Byzantine Generals Problem.” ACM TOPLAS, 1982.
- [26] S. Micali, M. Rabin, S. Vadhan. “Verifiable Random Functions.” FOCS 1999.
- [27] A. Miller, Y. Xia, K. Croman, E. Shi, D. Song. “The Honey Badger of BFT Protocols.” CCS 2016.
- [28] S. Nakamoto. “Bitcoin: A Peer-to-Peer Electronic Cash System.” Bitcoin Whitepaper, 2008.
- [29] A. Nemirovski, D. Yudin. *Problem Complexity and Method Efficiency in Optimization*. Wiley, 1983.
- [30] R. Pass, E. Shi. “The Sleepy Model of Consensus.” EUROCRYPT 2017.
- [31] Qubic Team. “Qubic: Quorum-Based Computation.” Qubic Documentation, docs.qubic.org, 2023.
- [32] A. Spiegelman, N. Giridharan, A. Sonnino, L. Kokoris-Kogias. “Bullshark: DAG BFT Protocols Made Practical.” CCS 2022.
- [33] Team Rocket, M. Yin, K. Sekniqi, R. van Renesse, E. G. Sirer. “Scalable and Probabilistic Leaderless BFT Consensus through Metastability.” arXiv:1906.08936, 2019.
- [34] E. B. Wilson. “Probable Inference, the Law of Succession, and Statistical Inference.” JASA, 1927.
- [35] G. Wood. “Polkadot: Vision for a Heterogeneous Multi-Chain Framework.” Polkadot Whitepaper, 2016.
- [36] D. Yin, Y. Chen, K. Ramchandran, P. Bartlett. “Byzantine-Robust Distributed Learning: Towards Optimal Statistical Rates.” ICML 2018.
- [37] M. Yin, A. Malkhi, M. K. Reiter, G. G. Gueta, I. Abraham. “HotStuff: BFT Consensus with Linearity and Responsiveness.” PODC 2019.